

基于需求预测与患者级多资源约束调度的超声预约优化

摘 要

超声预约同时受到多项目检查、设备专项能力、医生共享容量、患者准备和跨室转运的约束。本文以申请事件为决策单元，建立由需求预测、资源参数标定、纯住院调度和三类患者联合调度组成的患者级优化模型，并采用时间留出检验、精确子问题对照和独立可行性复核评价模型。

针对问题一，按患者与精确开单时刻聚合申请事件，将复合收费字段拆为项目任务，以训练期分位数划分平峰、高峰和压力状态。三类患者合计需求的平峰中位数为**412 例/日**，高峰和压力阈值分别为**671 例/日**与**797.15 例/日**；住院、门诊采用过去 8 周同星期中位数时，留出期 WAPE 分别为**16.04%**和**14.04%**。项目设备集合由题给设备说明逐项确定，床旁位置能力与项目能力取交集。

针对问题二，建立**5 分钟时间索引患者级调度模型**，同时决定患者顺序、项目次序、检查时刻和设备。针对全年规模，采用候选项目序列、共同设备优先和最早可行插入组成的确定性构造算法。FCFS-负荷均衡策略完整完成**60 368 例**住院事件，48 小时完成率为**85.30%**，等待时间中位数为 3.58 小时，因而作为纯住院推荐策略。

针对问题三，在统一设备与医生日历中建立 ε 约束联合调度模型，以背景服务率下限约束门诊、体检服务，并最大化住院 48 小时完成数。经验前沿的折中点为 $\alpha = 0.45$ ，完整完成住院**45 861 例**、背景**54 137 例**，对应完成率分别为**64.80%**和**56.37%**。该点表示标准班次和既定设备能力下的服务折中，医院可根据政策下限沿非支配前沿调整方案。

留出期预测、代表性子问题精确求解、排程结果独立复核以及参数敏感性共同检验模型。结果给出了峰平资源开放原则、患者级检查顺序和三类需求之间的可量化交换关系，可用于超声科预约与资源压力评估。

关键词：超声预约；需求预测；患者级调度；多资源约束； ε 约束

一、 问题重述

1.1 问题背景

某医院超声科同时接收住院、门诊和体检三类检查申请。一次申请可能包含多个项目，各项目在检查时长、准备条件和可用设备上存在差异；22 台设备具有明确的专项分工，医生又在不同设备之间共享。任一时刻，一台设备只能承担一项检查，同一患者的多个项目也不能并行执行。住院患者还受到开单后 48 小时完成全部项目的时限约束，门诊与体检需求则集中占用计划日内的服务容量。医疗预约研究通常需要同时处理等待、容量和服务水平 [1, 2]，分时预约和检查资源协同也是改善候检秩序的重要手段 [3, 4]，因此需要在设备能力、人员容量与患者完整流程之间进行统一配置。

1.2 问题重述

根据题目给出的历史检查记录、设备项目说明和医生信息，需要分析超声需求与资源状态，并针对纯住院场景及三类患者共享资源场景制定预约优化方案。具体需要完成以下三个问题：

(1) 分析住院、门诊和体检需求随日期、科室与项目的变化，识别平峰、高峰及压力状态，估计项目工时和医生并发能力，并据此给出设备开放、专项资源保留和预约分流原则。

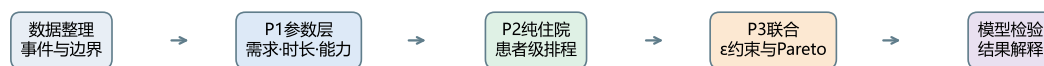
(2) 仅考虑住院患者，在 48 小时内完整完成全部检查的口径下，确定患者主序、项目次序、检查时刻和所用设备，使完成事件数尽可能多，并输出患者级预约与检查顺序。

(3) 在问题二的资源约束基础上加入门诊和体检患者，建立三类患者联合调度模型，刻画背景服务率与住院 48 小时完成率之间的交换关系，选择可实施的折中方案并检验其稳定性。

二、 问题分析

三个问题构成递进关系，整体建模链见图 1。问题一识别需求节律并确定项目工时、设备能力与医生容量；这些参数构成问题二纯住院调度的可行域。问题三在同一可行域中加入门诊和体检需求，由此衡量共享资源下不同患者群体之间的服务权衡。参数估计与结果评价按时间分开，以便考察需求规律对后续日期的适用性。

统一患者—项目—设备—医生排程建模链



训练期估计参数 (至2024-03-31) → 留出期仅评价 (2024-04-01至2025-03-31)

图 1 统一患者—项目—设备—医生排程建模链

2.1 问题分析

附件中的一行记录可能只是一次申请中的一个收费明细，单个医嘱字段也可能包含多个检查项目。数据处理需要建立“申请事件—项目任务”两层结构：先按患者编号与精确开单时刻聚合事件，再按明确边界拆分项目。完成率以事件为分母，工作量以项目任务为基础；报告和床旁加收等不占用扫描设备的记录不计入项目工时。

需求同时具有长期趋势、星期效应、月份效应和体检批次波动。采用训练期日需求分位数定义平峰、高峰与压力状态，并比较 7 日季节朴素、过去 8 周同星期中位数和日历趋势对数岭回归，以按时间向前的留出检验选择适合各类患者的预测方法。

题目没有给出扫查开始与结束时刻，报告间隔混合了候检、扫查、书写和提交过程，因此项目工时采用题给区间，历史相邻报告间隔只辅助比较项目族的相对负荷。候选设备则依据题目表 1 的项目描述逐项建立，使设备异质性以离散兼容集合进入后续调度。

2.2 问题二分析

问题二以患者完整完成为评价对象。某一项目未排入或超过 48 小时，整个住院事件即未达标，因此同一患者的任务序列需要整体安排。多项目患者既可能在同一设备连续完成，也可能跨设备完成；共同设备可减少转运，跨设备则扩大可行范围。患者顺序、项目顺序、开始时刻与设备选择相互影响，属于带时间窗和多资源约束的离散调度问题[5, 6, 7]。

全年时间索引整数规划的变量规模约为“患者数 $\times$ 项目数 $\times$ 设备数 $\times$ 时隙数”。据此先用 0-1 规划刻画完整可行域，再采用患者级构造算法搜索大规模解：生成少量具有业务含义的项目序列，按照设备稀缺性和准备条件确定尝试次序，在连续日历上寻找最早可行位置。由此得到患者级检查顺序和完成指标，并在从留出期抽取的代表性子问题中用 CP-SAT 精确模型评价构造解质量。

2.3 问题三分析

问题三需要在同一资源系统中处理住院、门诊和体检需求。题目未给出门诊、体检的最低完成率，也缺少真实预约日期，因此以报告日期构造回顾性计划日压力场景，用背景计划日完整完成率 R_B 描述服务水平。给定下限 α 后，在全评价期约束 $R_B \geq \alpha$ 并最大化住院 48 小时完成数，可形成一组可比较的 ε 约束解。

联合调度的难点是全评价期背景下限、住院 48 小时截止与逐时资源竞争同时存在。三类患者共享设备日历和医生并发容量，跨月任务仍在同一连续时间轴内处理；留出边界前后各保留 2 日以覆盖 48 小时任务。对不同 α 比较住院率、背景率、等待与资源负荷，再通过非支配筛选和理想点距离选择折中方案。

三、 模型假设

1、假设研究期内设备的可用状态和项目资质保持稳定，不考虑临时故障、维修停机及临时关闭诊室。

2、假设单个检查项目一旦开始便连续完成，不允许中途抢占，且项目不得跨越午休或日界。

3、假设每个检查项目同时占用一台兼容设备和一名医生；设备为独占资源，医生供给可用训练期星期与 5 分钟时隙的总并发容量近似。

4、假设同一患者的不同项目不能同时进行；相邻项目更换设备时需要 10 分钟转运，同设备连续检查不增加转运时间。

5、假设题目给出的项目时长区间能够代表标准检查工时，名义检查取区间中点，题设 5% 复杂检查取区间上界。

6、假设空腹、憋尿等准备要求按对应项目施加。空腹和饮水憋尿属于实际超声检查流程[8]；在附件缺少患者准备记录条件下，操作上设空腹项目在 10:00 前完成，憋尿项目在开单后至少准备 60 分钟。

7、假设床旁表示检查地点能力，床旁任务可用设备为项目兼容设备集合与 7 号床旁机的交集。

8、假设门诊、体检报告日期可作为回顾性计划日的压力代理，用于构造缺少真实预约日期时的历史需求场景。

9、假设患者最后一个排程项目的结束时刻可近似表示报告完成时刻，该近似仅用

于预约与容量评估。

四、 符号说明

表 1 主要符号

符号	含义	单位/取值
$\mathcal{I}$	全部患者申请事件集合	集合
$\mathcal{I}_I$	住院患者申请事件集合	集合
$\mathcal{I}_B$	门诊、体检背景事件集合	集合
$\mathcal{J}_i$	事件 i 包含的检查项目集合	集合
$\mathcal{R}$	超声设备或对应诊室集合	22 台
$\mathcal{T}$	离散时间槽集合	5 分钟/槽
$\mathcal{D}$	调度日期集合	日
$\mathcal{A}$	医生集合	人
p_i	事件 i 的医嘱下达时刻	时间戳
r_i	事件 i 按时间网格与计划窗口确定的最早可服务时刻	时间戳
d_i	事件 i 的完成截止时刻	时间戳
δ_i	背景事件 i 的计划日代理	日
$\pi(i)$	事件 i 对应的真实患者标识	标识
τ_{ij}	项目 (i, j) 的检查时长	分钟
ℓ_{ij}	项目时长向上取整后的槽数	槽
$\mathcal{R}_{ij}^{\text{final}}$	项目 (i, j) 的最终兼容设备集合	集合
$C_{w,t}$	星期 w 、槽 t 的医生并发容量	人
$g(r, r')$	相邻项目由 r 转至 r' 的转运时间	分钟
b_{ij}	项目 (i, j) 是否要求床旁检查	0–1
x_{ijrt}	项目 (i, j) 是否在设备 r 、槽 t 开始	0–1
y_i	事件 i 是否完整且按时完成	0–1
s_{ij}	项目 (i, j) 的开始时刻	时间戳
q_{ij}	项目 (i, j) 所选设备	设备
$z_{ij,k\ell}$	同一患者的任务 (i, j) 是否先于 (k, ℓ)	0–1
R_I	住院 48 小时完整完成率	[0, 1]
R_B	门诊、体检计划日代理完成率	[0, 1]
α	问题三设定的背景服务率下限	[0, 1]
$D(\alpha)$	前沿点到归一化理想点的距离	无量纲

五、 模型的建立与求解

5.1 问题一模型的建立与求解

5.1.1 事件一项目两层数据结构

设原始明细记录为 k 。先统一患者、开单、报告、项目、医生和设备字段，时间精确到秒。对同一来源、同一患者编号和同一开单时间的记录聚合为事件 i ：

$$i = (\text{source}, \text{patientID}, \text{orderTime}). \quad (1)$$

若医嘱字段形如 $[a, \text{彩超}] [b, \text{彩超}]$ ，则按方括号边界拆成两个项目任务，而不按普通逗号拆分，避免把“双侧下肢动脉（股总、股浅、股深等）”错误拆成多个项目。拆分后

删除“床旁 B 超检查加收”“超声计算机图文报告”等非扫查项；床旁字样仍保存在事件和项目标记中。

一个事件的全部有效项目构成集合 $\mathcal{J}_i$ 。其名义工作量和复杂病例工作量分别为

$$W_i^{(0)} = \sum_{j \in \mathcal{J}_i} \tau_{ij}^{(0)}, \quad (2)$$

$$W_i^{(1)} = \sum_{j \in \mathcal{J}_i} \tau_{ij}^{(1)}. \quad (3)$$

其中 $\tau^{(0)}$ 取题给区间中点， $\tau^{(1)}$ 取上界。连续模拟包含 167 361 个事件、328 950 个项目任务，具体范围见表 2。住院在留出边界前保留 258 个预热事件，门诊、体检留出期背景事件为 96 046 个。

表 2 连续模拟与留出期数据规模

来源	连续模拟事件	项目任务	留出期评价事件
住院	71,029	141,429	70,771
门诊	55,512	69,756	55,226
体检	40,820	117,765	40,820
合计	167,361	328,950	166,817

5.1.2 科室需求与多项目结构

临床科室需求量按训练期申请事件统计。对同一事件包含多个科室字段的极少数情况，以源记录顺序中的首个有效检查项目归属科室；未知科室单独保留，不向高频科室摊派。表 3 中的科室集中度取自训练期，多项目比例取自全观察期。住院需求分散于 87 个已知科室，前 6 科室只占 35.45%；门诊前 6 科室占 62.95%，适合按重点科室的星期节律预留普通检查容量；体检数据只对应一个体检科室，但其批次规模变化大，不能用“单一科室”等同于“需求稳定”。

表 3 训练期科室需求集中度与全观察期多项目事件比例

来源	训练期事件数	已知科室数	前 6 科室占比	全期多项目占比
住院	297,656	87	35.45%	51.17%
门诊	245,614	133	62.95%	19.72%
体检	239,570	1	100.00%	68.99%

住院和体检的多项目事件比例分别为 51.17% 和 68.99%，明显高于门诊的 19.72%。因此设备配置不能只按单项目数量配额：体检批次虽可集中安排，却更需要能够覆盖多个常见部位的共同设备；住院患者则需同时考虑专项能力和减少往返。题面没有给出同次检查固定节约比例，主模型仍逐项目累加题给工时。全观察期边界分析中，若将多项目事件总工时统一乘 0.9 或 0.8，住院平均名义工作量会由 17.39 分钟降至 16.11 或 14.83 分钟；这种节约缺乏项目组合层面的测时依据，故只用于说明容量范围，不进入排程参数。

训练期项目名称按互斥优先级分类。产科 III/IV 级、产科 I/II 级和心脏项目使用题给专用时长；血管、腹部/胃肠、泌尿/经腹妇科、经阴道/腔内、浅表器官、儿科专项、介入/定位和一般其他采用题给其他项目区间。先判别专项和检查路径，再判别一般部位，可避免“胎儿心脏”被普通心脏规则覆盖、“经阴道妇科”被经腹妇科覆盖。最终参数见表 4。

表 4 服务项目时长与训练期相邻报告代理

项目族	下界/min	名义/min	上界/min	训练代理数
产科 III/IV 级	30	35	40	13,244
产科 I/II 级	15	17.5	20	20,070
心脏	10	12.5	15	140,999
介入/定位	5	7.5	10	6,767
血管	5	7.5	10	58,868
经阴道/腔内	5	7.5	10	21,233
泌尿/经腹妇科	5	7.5	10	74,934
腹部/胃肠	5	7.5	10	99,686
浅表器官	5	7.5	10	67,810
儿科专项	5	7.5	10	3,949
一般其他	5	7.5	10	3,397

5.1.3 需求趋势与峰平阈值

对来源 c 和自然日 d ，定义申请事件量

$$N_{c,d}^{\text{ord}} = |\{i : \text{source}_i = c, \lfloor p_i \rfloor = d\}|, \quad (4)$$

报告事件量 $N_{c,d}^{\text{rep}}$ 则以事件报告日期的众数归日。报告量可反映历史完成节奏，但问题二、三的未来需求只能由申请量驱动。

2019–2024 年住院开单日均值持续上升：2020 年为 146.12 例/日，2021 年 161.47 例/日，2022 年 170.47 例/日，2023 年 178.02 例/日，2024 年 194.44 例/日。体检呈明显批次性和年份波动，门诊相对平稳。图 2 显示三类需求在星期与月份结构之外还存在批量体检与偶发需求集中，因而仅用总体均值不足以设计容量。

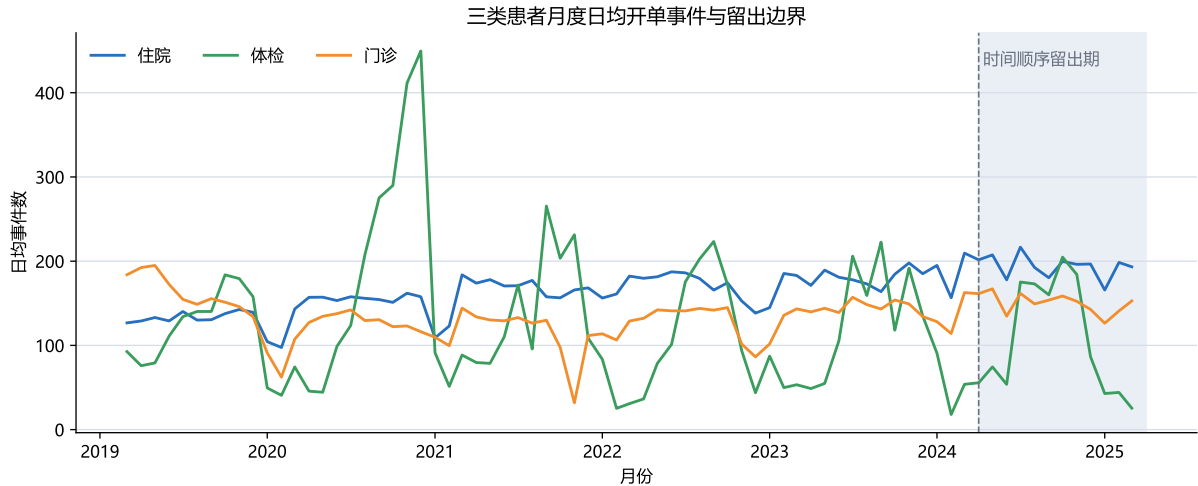


图 2 三类患者月度日均开单事件与留出边界

所有运行状态阈值只由训练期确定。对训练期完整日需求序列，定义平峰区间为

$[Q_{0.45}, Q_{0.55}]$ ，高峰阈值为 $Q_{0.90}$ ，压力阈值为 $Q_{0.95}$ ：

$$\mathcal{D}_{\text{flat}} = \{d : Q_{0.45} \leq N_d \leq Q_{0.55}\}, \quad (5)$$

$$\mathcal{D}_{\text{peak}} = \{d : N_d \geq Q_{0.90}\}, \quad (6)$$

$$\mathcal{D}_{\text{stress}} = \{d : N_d \geq Q_{0.95}\}. \quad (7)$$

代表日取最接近相应阈值且时间最早的日期，保证规则可复现。三类开单合计的 $Q_{0.45}$ 、平峰中位、 $Q_{0.55}$ 分别为 388.65、412 和 434 例/日，高峰阈值 671 例/日，压力阈值 797.15 例/日。训练期峰日代表为 2019 年 11 月 6 日，共 671 例；平峰代表为 2022 年 3 月 2 日，共 412 例。留出期超过训练高峰阈值的天数占 13.15%，超过压力阈值占 7.95%，说明容量压力不是由少量异常值造成。

5.1.4 时间顺序需求检验

为考察需求规律能否外推，建立三类简单模型。7 日季节朴素为

$$\hat{N}_d = N_{d-7}. \quad (8)$$

过去 8 周同星期中位数为

$$\hat{N}_d = \text{median}\{N_{d-7}, N_{d-14}, \dots, N_{d-56}\}. \quad (9)$$

日历趋势模型在训练期拟合

$$\begin{aligned} \log(1 + N_d) = & \beta_0 + \beta_1 t_d + \beta_2 \sin \frac{2\pi h_d}{365.25} + \beta_3 \cos \frac{2\pi h_d}{365.25} \\ & + \sum_{w=1}^6 \gamma_w I(w_d = w) + \sum_{m=2}^{12} \eta_m I(m_d = m) + \varepsilon_d. \end{aligned} \quad (10)$$

其中 h_d 为年内日序。模型用正余弦项刻画年周期，并以岭惩罚控制月份和星期虚拟变量的共线性。预测回到原尺度时使用 $\exp(\hat{z}) - 1$ ，避免把对数尺度预测直接当作人数。

评价指标为

$$\text{WAPE} = \frac{\sum_d |N_d - \hat{N}_d|}{\sum_d N_d}, \quad (11)$$

$$\text{sMAPE} = \frac{1}{n} \sum_d \frac{2|N_d - \hat{N}_d|}{|N_d| + |\hat{N}_d|}. \quad (12)$$

在严格时间留出期内，住院开单由过去 8 周同星期中位数取得最低 WAPE 16.04%，门诊开单为 14.04%；体检开单因批量计划变化，最好的日历趋势模型 WAPE 仍为 69.83%。三类合计由日历趋势模型取得 24.92%。这说明住院、门诊的星期规律较稳定，而体检量需要未来批次计划才能获得高精度预测。既有医院门诊量研究也观察到趋势、周内效应和序列相关性 [9, 10, 11]；后续排程使用实际留出期需求做回放评价，将需求预测误差与调度误差分开。

表 5 开单日需求预测的留出期 WAPE 比较

来源	7 日季节朴素	8 周同星期中位数	日历趋势岭回归
住院	20.65%	16.04%	16.50%
门诊	19.45%	14.04%	22.16%
体检	106.43%	74.94%	69.83%
三类合计	35.06%	25.19%	24.92%

表 5 说明模型选择按来源分别进行，而不是强制三类需求共用一个预测器。住院和门诊的 8 周同星期中位数能吸收近期水平变化，又不依赖未来数据；体检的三种误差都较大，根源是团检计划具有外生批次信息。实际部署时，住院、门诊可由历史滚动预测，体检必须接入未来团检名单；若名单不可得，应以高分位压力场景而非单点预测配置容量。

5.1.5 项目时长与医生效率边界

历史报告时间不能直接给出扫查时长。数据驱动的超声检查时长估计需要独立的服务过程字段 [12]。本文只对同一医生同一报告日的相邻单项目事件计算正间隔，并截取 $(0, 60]$ 分钟作为局部吞吐代理。有效单项目代理共 510 957 个，中位数 7 分钟；代理同时包含扫查、书写、等候和批量提交，因此只用于检查题给时长的相对排序。例如产科 III/IV 级代理中位数 21 分钟，高于一般项目 5–8 分钟，支持其更高负荷，但主排程仍使用题给 30–40 分钟而非 21 分钟。

对医生 a 在日期 d 按题给名义时长汇总的工作量 L_{ad} 和有效活动窗口 H_{ad} ，构造对数生产率

$$z_{ad} = \log \frac{L_{ad}}{H_{ad}}. \quad (13)$$

其中， H_{ad} 取当日首末报告间隔再加首个事件的名义工时；仅保留不少于 3 个事件且首末跨度不超过 16 小时的医生日。对 z_{ad} 作 1% 和 99% 温莎化后，记医生 a 的样本均值、均值方差和医生间方差分别为 $\bar{z}_a$ 、 $\hat{v}_a$ 和 σ_b^2 ，采用经验贝叶斯收缩

$$\tilde{z}_a = \lambda_a \bar{z}_a + (1 - \lambda_a) \mu_0, \quad (14)$$

$$\lambda_a = \frac{\sigma_b^2}{\sigma_b^2 + \hat{v}_a}. \quad (15)$$

该指标只描述历史报告吞吐差异，不映射到未来 33 名医生。主排程中的供给参数来自表 6 工作日记录：将训练日首末活动时刻重构为活动区间，按星期和 5 分钟槽统计同时活动人数，主情景取中位数，偏紧情景取下四分位。设经验分位容量为 $\hat{C}_{w,t}$ ，题面医生数为 33、设备数为 22，则实际共享并发容量取

$$C_{w,t} = \min\{\hat{C}_{w,t}, 33, 22\}. \quad (16)$$

这一双上限反映每项检查同时占用一名医生和一台设备；训练中位容量实际为 5–20，如图 3。

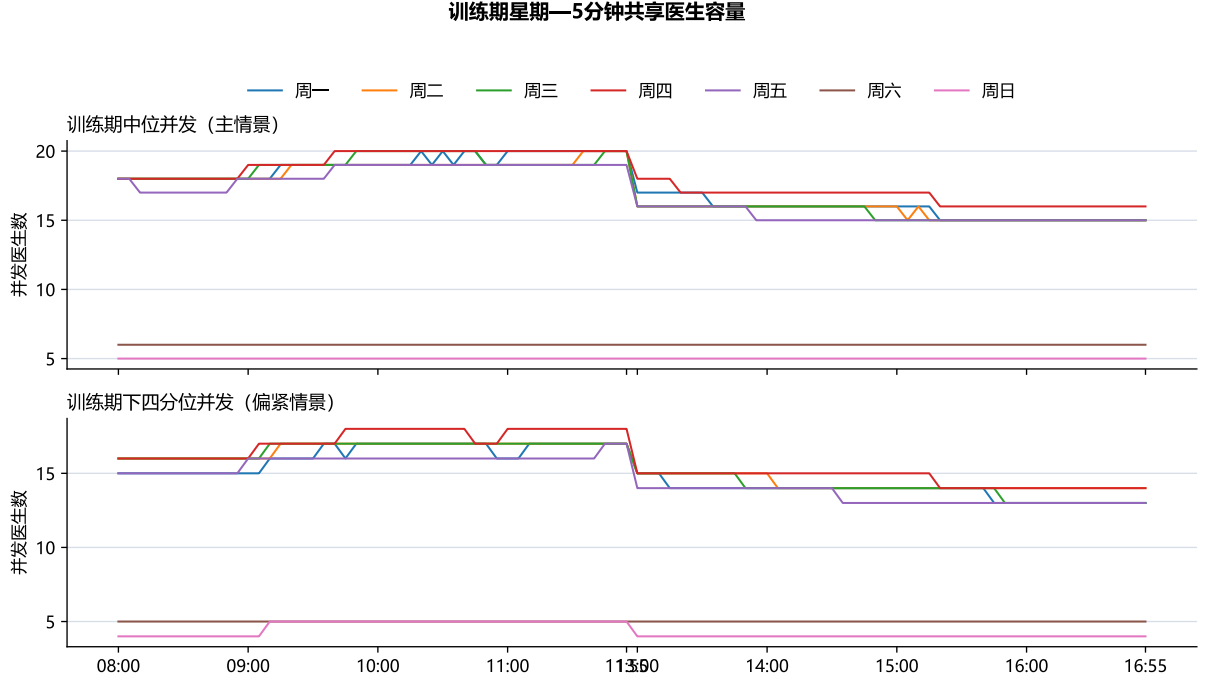


图 3 训练期星期—5 分钟共享医生容量

5.1.6 项目—设备兼容模型

设表 1 中设备 r 的项目描述集合为 $\mathcal{E}_r$ 。对规范化项目文本 v_j ，定义匹配类型

$$m(j, r) \in \{0, 1, 2, 3\}. \quad (17)$$

其中 1 表示项目文本直接命中；2 表示保留解剖部位和检查路径的同义或包含关系，复合项目取组成部位的同设备交集；3 只对应表 1 明确的广义描述，例如“带有床旁的超声项目”对常规腹部、泌尿和胸水床旁任务的有限覆盖；0 表示表 1 不能确认。主模型兼容集为

$$\mathcal{R}_{ij} = \{r : m(j, r) \in \{1, 2, 3\}\}. \quad (18)$$

对规范化后的 211 个项目名称逐项匹配表 1，其中 44 个直接命中，60 个通过明确的同义或包含关系匹配，92 个由题面广义描述覆盖，另有 15 个无法确认。最后一类任务仍保留在事件分母中，但候选设备集为空。匹配覆盖见图 4 和表 6。

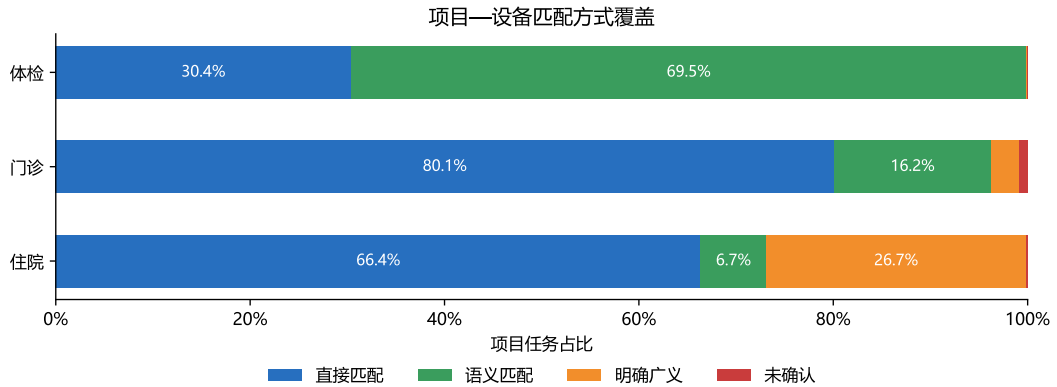


图 4 项目—设备匹配方式覆盖

表 6 项目—设备匹配方式统计

匹配方式	项目数	项目任务数	使用规则
直接匹配	44	185,497	表 1 项目文本直接命中
语义匹配	60	102,651	同义、包含或复合部位交集
明确广义	92	39,893	表 1 中的广义项目描述覆盖
未确认	15	909	表 1 无法确认，主模型排除

床旁条件单独处理。令 $b_{ij} = 1$ 表示项目层床旁任务，7 号机集合为 $\mathcal{R}^{\text{bed}} = \{7\}$ ，则

$$\mathcal{R}_{ij}^{\text{final}} = \begin{cases} \mathcal{R}_{ij} \cap \mathcal{R}^{\text{bed}}, & b_{ij} = 1, \\ \mathcal{R}_{ij}, & b_{ij} = 0. \end{cases} \quad (19)$$

该定义使“床旁”只改变地点，不改变专项能力。床旁心脏、床旁产科和床旁血管等任务仅在 7 号机具备相应项目能力时进入候选集合。

5.1.7 峰平状态下的机器与预约配置

项目一设备集合给出“哪些机器能够做”，峰平配置进一步回答“何时开放、先安排什么”。表 7 以训练期代表日给出三类构成。平峰代表日虽总量为 412 例，但住院占 212 例，不能被简单理解为低负荷日；高峰代表日的 671 例主要来自 321 例体检批次，调度方式应与住院高峰不同。因而调整以来源结构和专项负荷共同决定，不按总人数同比例放大每台机器。

表 7 训练期峰平状态及对应资源配置原则

状态	合计/日	住院	门诊	体检	机器与预约配置
平峰代表日	412	212	161	39	专项项目固定进入相容诊室；综合项目按最早空闲分配，多项目优先共同设备。
高峰代表日	671	163	187	321	同时开放数不超过时隙医生容量（最高 20）；保留专项机容量，综合机承担可替代项目，体检批次分时进入。
压力状态	≥ 797.15	—	—	—	标准班次内先满足政策服务下限；其余背景预约移日或扩展工作时段，不改变设备项目资质。

设 $K_{w,t}^{\text{need}}$ 为预测任务在星期 w 、时隙 t 所需的并行诊室数，则建议开放数为

$$K_{w,t}^{\text{open}} = \min\{C_{w,t}, K_{w,t}^{\text{need}}\}. \quad (20)$$

主情景的医生并发上限为 20，因此开放设备数随时隙容量变化，不采用 22 台设备全天同时运行的固定设置。平峰状态下，先开放当日实际有专项需求的心脏、产科、血管、腹部和床旁设备，再由综合设备补足 $K_{w,t}^{\text{open}}$ ；门诊与体检按时段分批，多项目患者优先在共同设备连续完成。高峰状态下，同时开放数可提高至医生容量上限 20，专项设备保留给不可替代项目，综合设备承接兼容项目，体检批次错开住院释放集中的时段。压力状态下，现有标准班次先执行既定服务下限；超出能力的可预约背景需求移日，或在医院批准后延长工作时段。三种状态均保持项目设备资质不变，由此形成需求预测、专项资源保留、普通资源弹性开放和背景预约分流相结合的配置方案。

5.2 问题二模型的建立与求解

5.2.1 排程对象与完成口径

问题二只加载住院事件，不预占任何门诊、体检容量。记 p_i 为医嘱下达时刻， r_i 为 p_i 向上取整到 5 分钟后在工作日历中的最早可服务时刻；48 小时截止时刻为

$$d_i = p_i + 48 \text{ h}. \quad (21)$$

若所有 $j \in \mathcal{J}_i$ 均在 d_i 前结束，则 $y_i = 1$ ，否则 $y_i = 0$ 。住院 48 小时完整完成率为

$$R_I = \frac{\sum_{i \in \mathcal{I}_I^{\text{eval}}} y_i}{|\mathcal{I}_I^{\text{eval}}|}. \quad (22)$$

留出期评价集合包含 70 771 个事件。258 个 3 月 30–31 日预热事件可占用 4 月初资源，但既不进入分子也不进入分母；3 月 31 日事件的截止期可延伸到 4 月 2 日，故日历尾端保留 2 日而不按月重置。

5.2.2 时间索引整数规划

令 $x_{ijrt} = 1$ 表示事件 i 的项目 j 在设备 r 、时隙 t 开始， $a_{ijrtu} = 1$ 表示该选择会占用时隙 u 。时长以 5 分钟向上取整：

$$\ell_{ij} = \left\lceil \frac{\tau_{ij}}{5} \right\rceil. \quad (23)$$

事件采用“全部安排或全部不安排”的完整性约束：

$$\sum_{r \in \mathcal{R}_{ij}^{\text{final}}} \sum_{t \in \mathcal{T}_{ijr}} x_{ijrt} = y_i, \quad \forall i, j \in \mathcal{J}_i. \quad (24)$$

由此，事件被选择时每个项目恰好安排一次，未被选择时不占用任何资源。

设备互斥约束为

$$\sum_i \sum_j \sum_{t: a_{ijrtu}=1} x_{ijrt} \leq 1, \quad \forall r, u. \quad (25)$$

同一患者不能同时检查，且同一患者在不同时刻开出的多个申请事件仍共享同一患者标识 $\pi(i)$ ：

$$\sum_{i: \pi(i)=v} \sum_j \sum_r \sum_{t: a_{ijrtu}=1} x_{ijrt} \leq 1, \quad \forall v, u. \quad (26)$$

医生是所有设备共享的并发资源。设时隙 u 对应星期 $w(u)$ ，训练期容量为 $C_{w(u),u}$ ，则

$$\sum_i \sum_j \sum_r \sum_{t: a_{ijrtu}=1} x_{ijrt} \leq C_{w(u),u}, \quad \forall u. \quad (27)$$

这比“每个房间各有若干医生”的假设更符合题面，因为 33 名医生应在 22 台设备之间共享。

释放与截止通过可选开始集合 $\mathcal{T}_{ijr}$ 保证：

$$t \geq r_i, \quad (28)$$

$$t + 5\ell_{ij} \leq d_i. \quad (29)$$

任一项目不得跨越 12:00–13:00 午休或日界。对空腹项目，进一步要求

$$t + 5\ell_{ij} \leq 10:00. \quad (30)$$

对憋尿项目，项目级最早开始为

$$t \geq \lceil p_i + 60 \min \rceil_{5\min}. \quad (31)$$

床旁项目的设备集合已由 $\mathcal{R}_{ij} \cap \{7\}$ 给出。

为使项目相对顺序成为决策，记 $X_{ij} = \sum_{r,t} x_{ijrt}$ 。对任意两个不同任务 (i, j) 与 (k, ℓ) ，若 $\pi(i) = \pi(k)$ ，令 $z_{ij,kl} = 1$ 表示前者先于后者。仅当两个任务均被选中时，必须且只能选择一个方向：

$$z_{ij,kl} + z_{kl,ij} \leq X_{ij}, \quad (32)$$

$$z_{ij,kl} + z_{kl,ij} \leq X_{kl}, \quad (33)$$

$$z_{ij,kl} + z_{kl,ij} \geq X_{ij} + X_{kl} - 1. \quad (34)$$

设备切换的转运间隔定义为

$$g(q_{ij}, q_{kl}) = \begin{cases} 0, & q_{ij} = q_{kl}, \\ 10 \min, & q_{ij} \neq q_{kl}. \end{cases} \quad (35)$$

利用足够大的常数 M 关闭未选方向，成对非重叠与转运约束为

$$s_{kl} \geq s_{ij} + 5\ell_{ij} + g(q_{ij}, q_{kl}) - M(1 - z_{ij,kl}), \quad (36)$$

$$s_{ij} \geq s_{kl} + 5\ell_{kl} + g(q_{kl}, q_{ij}) - M(1 - z_{kl,ij}). \quad (37)$$

时间索引变量决定开始时刻，顺序变量表达同一患者在事件内及跨事件的成对先后关系；构造算法则在安排每个任务时同步检查该患者已占用的全部区间。问题二的主目标为

$$\max \sum_{i \in \mathcal{I}^{\text{eval}}} y_i. \quad (38)$$

完成数相同时，再按等待时间和跨室等运营指标选择推荐策略，不用次级指标交换一个完整完成事件。对完整排入的事件，等待时间从最早可服务时刻计至首项检查开始：

$$w_i = \min_{j \in \mathcal{J}_i} s_{ij} - r_i, \quad y_i = 1. \quad (39)$$

人员在岗容量利用率按全部已排任务分钟占可用医生分钟的比例计算：

$$U_{\text{staff}} = \frac{\sum_{i,j,r,t} 5\ell_{ij} x_{ijrt}}{5 \sum_{u \in \mathcal{T}} C_{w(u),u}}. \quad (40)$$

多项目跨室率以评价期全部多项目住院事件为分母；完整排入且至少发生一次设备切换的事件进入分子，未完整排入者不产生实际跨室。因此该指标用于比较患者移动负担时，需要与完整完成率同时考察。

5.2.3 大规模构造算法

全年时间索引模型过大，本文保留上述可行域，改用确定性患者级构造算法求解。诊断资源的动态优先级和住院患者柔性排程已有类似建模思想 [13, 14]，本文算法内核

由“候选序列—共同设备—最早可行插入”组成。

步骤 1：候选项目序列。对每个事件生成四类确定性序列：空腹优先依次按兼容设备数少、时长排序；空腹优先依次按时长、兼容设备数少排序；兼容设备数少后按空腹优先排序；以及原始项目序。重复序列只保留一次，所有序列均通过患者互斥约束执行。

步骤 2：共同设备优先。先计算

$$\mathcal{R}_i^\cap = \bigcap_{j \in \mathcal{J}_i} \mathcal{R}_{ij}^{\text{final}}. \quad (41)$$

若 $\mathcal{R}_i^\cap \neq \emptyset$ ，先按相应设备规则排序，取前三个共同设备，并用“空腹优先、设备选择少者优先、长项目优先”的首选项目序列尝试同室完成；这些尝试均失败后，再枚举其余候选序列并允许跨室。所有策略均先选取最早可行时刻；当多个设备同样早时，分别比较设备编号、当前累计负荷或训练期稀缺权重

$$\omega_r = \sum_j \frac{D_j \tau_j}{|\mathcal{R}_j|} \mathbb{I}(r \in \mathcal{R}_j), \quad (42)$$

由此形成最低编号、负荷均衡和稀缺能力保留三种设备破同序方式，最后一种用于减少普通任务对高需求专项设备的占用。

步骤 3：连续日历搜索。对候选序列逐任务向前搜索。某一插入位置可行，当且仅当设备区间空闲、每个 5 分钟槽的医生负载未达上限、项目在单一工作时段内、准备条件满足且最终不越过 d_i 。若有多个位置，先选择开始最早者；同样早时优先延续上一设备，再按相应候选策略的设备编号、累计负荷或稀缺权重破同序。

步骤 4：整体可行性确认。只有所有项目都找到位置，才确认该事件占用的设备与医生时段；中途失败则取消本次尝试的全部暂存位置，不保留部分任务。该规则与患者完整完成口径一致。

步骤 5：患者优先级。令 $\xi_i = 1$ 表示事件 i 被抽为题设复杂病例，主情景总工时为

$$W_i^* = (1 - \xi_i)W_i^{(0)} + \xi_i W_i^{(1)}. \quad (43)$$

比较两种住院队列：FCFS 按 r_i 和事件编号排序；最小松弛度保护按

$$\text{slack}_i = (d_i - p_i) - W_i^* \quad (44)$$

升序，并依次用 r_i 、候选设备数、截止时刻和事件编号破同序。 $d_i - p_i$ 表示从医嘱下达至截止的可用时长。两种患者顺序分别与最低编号、负荷均衡和稀缺能力保留三种设备规则组合，共形成六个候选策略；六者使用相同的项目序列、准备条件与可行性内核。对策略集合 $\mathcal{S}_2$ ，固定选择规则为

$$S_2^* = \arg \min_{S \in \mathcal{S}_2} \left(-N_I(S), P_{90}(S), P_{50}(S), r_{\text{switch}}(S), M(S), \text{id}(S) \right). \quad (45)$$

其中 N_I 为住院完整完成数， M 为排入总分钟，最后一项仅用于稳定破同序。负号保证任何次级指标都不以减少一个完整事件为代价。

5.2.4 问题二求解结果

表 8 比较两种患者队列与三种设备破同序方式组成的六个策略，图 5 进一步对照其完成率与条件等待。FCFS-负荷均衡完整完成 60 368 个留出期住院事件，完成率 85.30%，

在六个策略中最高；其条件等待 P50 和 P90 分别为 3.58 小时和 22.08 小时。按照式（45）的词典序规则，推荐 FCFS–负荷均衡策略。表中多项目需求跨室率采用前述固定分母，因而在六个策略间的排序等价于比较发生跨室的事件数。对比结果还表明，设备选择规则与患者队列共同影响可完成集合，不能只根据 FCFS 或松弛度标签判断策略优劣。

表 8 问题二纯住院策略比较

策略	48h 完成数	完成率	P50/h	P90/h	多项目需求跨室率
FCFS–最低编号	59,586	84.20%	25.50	47.50	57.85%
最小松弛度–最低编号	58,964	83.32%	20.25	46.00	60.76%
FCFS–负荷均衡	60,368	85.30%	3.58	22.08	58.16%
最小松弛度–负荷均衡	58,419	82.55%	3.33	21.67	57.29%
FCFS–稀缺能力保护	60,184	85.04%	6.08	29.42	57.83%
最小松弛度–稀缺能力保护	58,875	83.19%	6.42	29.67	58.08%

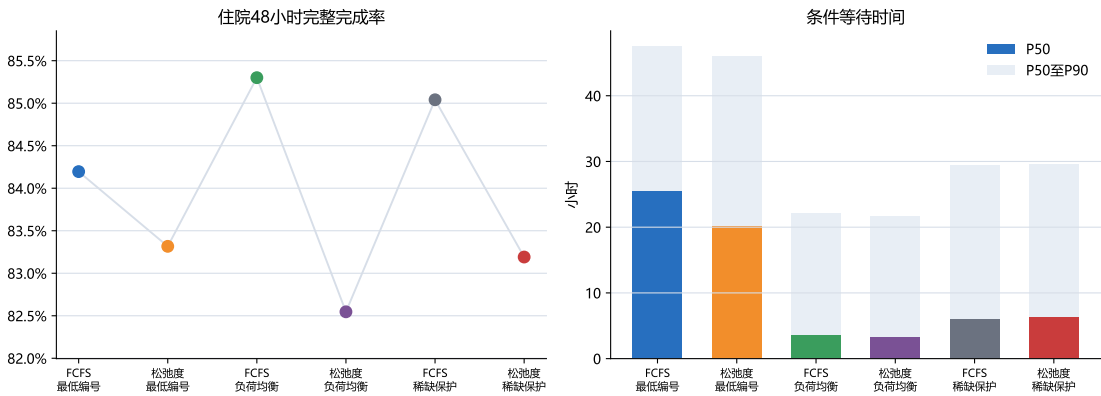


图 5 问题二住院完成率与条件等待比较

在评价分母中,10 403 个事件未能完整排入。表 9 按最终失败状态作互斥分解:10 258 个事件属于项目设备能力不满足,其中 222 个事件含有表 1 无法确认设备的项目,10 036 个事件的床旁任务在“项目兼容设备”与 7 号床旁机取交集后为空,其他 A–F 能力失败类别均为 0;另有 145 个事件虽各项目存在候选设备,但在 48 小时、准备窗口和已有负荷共同作用下没有形成完整计划。前一结果说明床旁地点能力与专项检查能力的交叉覆盖是主要瓶颈,不能用空闲普通诊室替代;医院若核定 7 号机具有更多专项能力,应先更新项目资质表再重新排程。

表 9 问题二评价期末未完成原因分解

未完成主因	事件数	占全部未完成
项目设备能力不满足（A–F 分解）	10,258	98.61%
具备项目能力但 48 小时内未形成完整计划	145	1.39%
合计	10,403	100.00%

完整完成者的 P90 等待为 22.08 小时、P95 为 23.67 小时。多项目患者中 58.16% 发生跨室，这一比例较高，原因是不同项目的专项设备集合交集不足；相邻项目更换实际诊室时均计入 10 分钟转运。

纯住院方案共排入 1 344 765 分钟，人员在岗容量利用率为 54.53%。这一总体利用率不能说明各机器均有富余：表 10 中排入分钟最高的 5 个诊室，其有任务日平均利用率约为 54.96%–54.97%，负荷均衡规则已使普通兼容任务较均匀地分布；与此同时，能力集合为空的任务即使遇到普通机器空闲也不能转入。床旁、血管、心脏和产科任务仍受各自候选集合限制。因此问题二的主要改进方向是核定真实项目资质并均衡相容任务，而不是只提高全科总开机率。

表 10 问题二排入分钟最高的诊室

诊室	排入分钟	有任务日平均利用率
门 6	96,840	54.97%
门 11	96,840	54.97%
门 7	96,830	54.97%
病 4	96,825	54.96%
门 5	96,825	54.96%
门 10	55,700	31.62%

问题二输出患者、项目、开始时刻、结束时刻和设备组成的推荐表，部分结果见表 11。每个项目独立保留，事件内序号给出实际执行顺序；相邻项目跨设备时，下一项目开始前已经计入转运。

表 11 患者级推荐排程样例

序位	事件	项目序位	项目族	时段	设备
1	IP2024-000259	1	腹部/胃肠	04-01 08:30–08:40	门 16(65)
2	IP2024-000261	1	浅表器官	04-01 08:30–08:40	妇门 2(75)
3	IP2024-000265	1	浅表器官	04-01 08:35–08:45	门 9(47)
4	IP2024-000268	1	血管	04-01 08:50–09:00	病 2(71)
4	IP2024-000268	2	血管	04-01 09:40–09:50	病 2(71)
5	IP2024-000267	1	浅表器官	04-01 09:00–09:10	病 3(72)
6	IP2024-000271	1	血管	04-01 09:00–09:10	门 3(42)
6	IP2024-000271	2	介入/定位	04-01 09:45–09:55	门 3(42)

综合 48 小时完成数、等待时间和跨室比例，最终选取 FCFS–负荷均衡作为住院策略。患者内部先处理候选设备较少和准备窗口较紧的项目，优先使用共同设备，无法同室完成时再按最早可行时刻跨室安排。推荐表由此同时给出患者顺序、项目次序、检查时刻和诊室；全年构造解的求解质量由第六章的精确子问题对照界定。

5.3 问题三模型的建立与求解

5.3.1 背景需求与服务指标

门诊和体检附件包含开单与报告日期，但缺少真实预约日期和历史预约时段。为构造可比较的回顾性压力场景，报告日期众数仅用于确定计划日 δ_i ；记 p_i 为医嘱下达时刻，背景事件最早可服务时刻不早于开单时刻的 5 分钟上取整和计划日 08:00，截止时刻为计划日 17:00：

$$r_i^B = \max\{\lceil p_i \rceil_{5\min}, \delta_i + 8\text{h}\}, \quad (46)$$

$$d_i^B = \delta_i + 17\text{h}. \quad (47)$$

历史报告具体时刻、历史设备和医生不进入调度。记 $\mathcal{I}_B^{\text{eval}}$ 为留出期门诊、体检事件集合，背景计划日完整完成率定义为

$$R_B = \frac{\sum_{i \in \mathcal{I}_B^{\text{eval}}} y_i}{|\mathcal{I}_B^{\text{eval}}|}. \quad (48)$$

该指标表示标准班次对历史计划日需求的承接能力，与真实历史准点率属于不同口径。

5.3.2 全局 ε 约束模型

问题三以住院 48 小时完成率 R_I 和背景服务率 R_B 为两个主要目标。三类患者在同一设备与医生日历上竞争资源，超声预约中的检查室指派可在统一模型中处理 [15]；医疗预约研究也表明，服务时间不确定时需要同时考虑预约分配与患者排序 [16]。给定背景服务下限 α ，全评价期最低背景完成数为

$$B_{\min}(\alpha) = \lceil \alpha |\mathcal{I}_B^{\text{eval}}| \rceil. \quad (49)$$

由此建立 ε 约束模型

$$\max \sum_{i \in \mathcal{I}_I^{\text{eval}}} y_i, \quad (50)$$

$$\text{s.t.} \quad \sum_{i \in \mathcal{I}_B^{\text{eval}}} y_i \geq B_{\min}(\alpha), \quad (51)$$

$$\text{问题二中的设备、医生、患者、准备、截止与转运约束成立.} \quad (52)$$

正式约束作用于全评价期，并未要求每个自然日完成固定比例。对候选下限集合 $\mathcal{A}_\alpha$ 逐点求解，达到相应下限的解进入非支配筛选，未达到下限的运行点只用于识别容量边界。

5.3.3 住院优先基线与全局缺口重构

联合求解与问题二共享患者级可行性内核，包括项目顺序候选、共同设备优先、最早可行插入和失败回滚。对给定 α ，先从空日历构造住院优先基线 $S_\alpha^{(0)}$ ，即依次排入住院和背景事件。若该基线已经达到全评价期背景下限，则直接保留；否则从空日历分别按“稀缺能力优先”和 FCFS 两种背景顺序重构候选解：先排入背景事件直至达到 $B_{\min}(\alpha)$ ，再排住院事件，最后用剩余容量继续排背景事件。该过程只使用全期累计目标，不设置逐日配额。

令 $B(S)$ 和 $I(S)$ 分别表示候选解 S 的背景与住院完整完成数，候选集合及可行集合为

$$\mathcal{C}_\alpha = \begin{cases} \{S_\alpha^{(0)}\}, & B(S_\alpha^{(0)}) \geq B_{\min}(\alpha), \\ \{S_\alpha^{(0)}, S_\alpha^{(\text{scar})}, S_\alpha^{(\text{fcfs})}\}, & \text{其他情形,} \end{cases} \quad (53)$$

$$\mathcal{F}_\alpha = \{S \in \mathcal{C}_\alpha : B(S) \geq B_{\min}(\alpha)\}. \quad (54)$$

为控制全年计算量，仅在基线未达到下限时生成两个重构候选。在 $\mathcal{F}_\alpha$ 中先最大化 $I(S)$ ，再依次最小化住院等待 P90、P50、多项目跨室率和排入分钟，并以稳定标识破同序。若 $\mathcal{F}_\alpha$ 为空，则记录背景完成数最高的候选以量化当前算法的下限缺口，但不将其作为该 α 下的可行点。由 $\mathcal{F}_\alpha = \emptyset$ 只能判断本次有限候选搜索未找到达标方案，不能据此判定数学模型不可行或断言系统的理论容量上界。

5.3.4 Pareto 前沿与方案选择

表 12 给出候选下限及联合排程结果，图 6 显示两类服务率之间的交换关系。 $\alpha = 0$ 对应背景无最低要求时的住院边界；提高 α 后，背景完成率上升，住院可用容量相应减少。达到服务下限且不被其他点同时支配的运行点构成经验 Pareto 前沿。

表 12 问题三背景下限网格及联合排程结果

α	背景完成数	背景率	住院完成数	住院率	P90/h
0.00	38,872	40.47%	60,368	85.30%	22.08
0.05	38,872	40.47%	60,368	85.30%	22.08
0.10	38,872	40.47%	60,368	85.30%	22.08
0.15	38,872	40.47%	60,368	85.30%	22.08
0.20	38,872	40.47%	60,368	85.30%	22.08
0.25	38,872	40.47%	60,368	85.30%	22.08
0.30	38,872	40.47%	60,368	85.30%	22.08
0.35	38,872	40.47%	60,368	85.30%	22.08
0.40	38,872	40.47%	60,368	85.30%	22.08
0.45	54,137	56.37%	45,861	64.80%	21.83
0.50	55,708	58.00%	44,439	62.79%	22.00
0.525	56,281	58.60%	43,852	61.96%	21.92
0.55	57,186	59.54%	43,069	60.86%	22.33
0.575	59,996	62.47%	42,440	59.97%	30.33
0.60	61,075	63.59%	41,684	58.90%	30.42
0.65	63,085	65.68%	40,273	56.91%	30.33

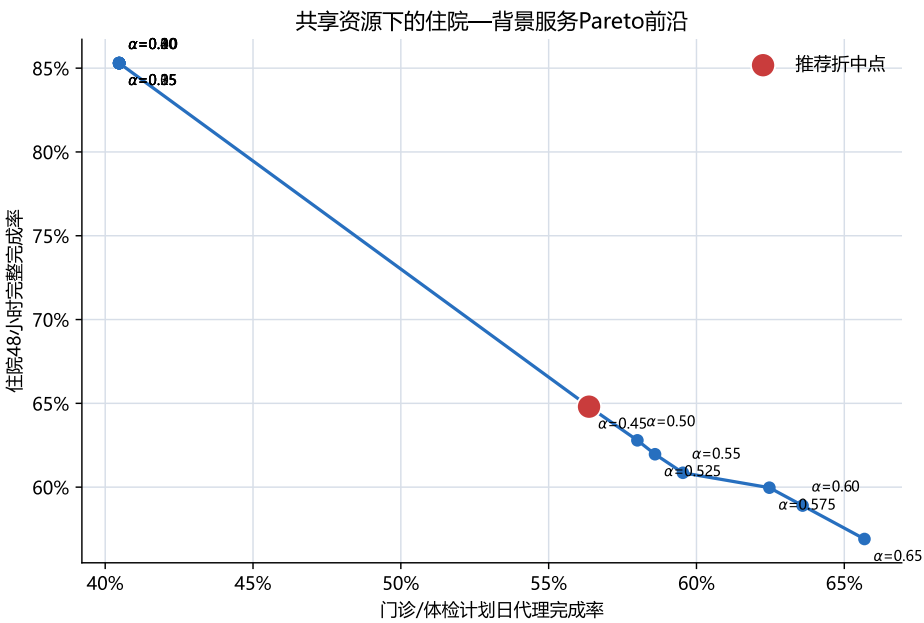


图 6 共享资源下的住院—背景服务 Pareto 前沿

设可达前沿的理想点为 $(R_B^{\max}, R_I^{\max})$ 。对每个非支配点计算归一化距离

$$D(\alpha) = \sqrt{\left(\frac{R_B^{\max} - R_B(\alpha)}{R_B^{\max} - R_B^{\min}}\right)^2 + \left(\frac{R_I^{\max} - R_I(\alpha)}{R_I^{\max} - R_I^{\min}}\right)^2}. \quad (55)$$

选择 $D(\alpha)$ 最小的点，并依次以背景率和住院率破同序。最终推荐 $\alpha = 0.45$ ：全期背景最低完成数为 43 221 例，所得方案完整完成住院 45 861 例、背景 54 137 例，即 $R_I = 64.80\%$ 、 $R_B = 56.37\%$ ；住院条件等待 P50、P90 和 P95 分别为 3.92、21.83 和 25.58 小时。该结果表示当前标准班次、设备能力和医生容量下的折中服务水平。

前沿的高背景服务点进一步说明政策下限的代价： $\alpha = 0.60$ 时背景率为 63.59%、住院率为 58.90%， $\alpha = 0.65$ 时二者分别为 65.68% 和 56.91%。 $\alpha = 0.70$ 要求至少完成 67 233 例背景事件，而当前构造算法完成 63 278 例，尚差 3 955 例，因此该运行点不进入可行前沿。这里的“未达标”是启发式搜索结果，不是模型数学不可行的证明；若医院提出 70% 背景下限，应先扩大候选搜索或采用滚动精确优化验证，在仍无法满足时再评估扩展班次和调整可预约需求。

表 13 纯住院与推荐联合方案的运行形态

运行指标	P2 纯住院	P3 推荐联合方案
排入检查分钟	1,344,765	1,897,045
人员在岗容量利用率	54.53%	76.93%
住院等待 P95/小时	23.67	25.58
多项目需求跨室率	58.16%	30.63%
背景全期下限/完成数	—	43,221/54,137

表 13 给出资源竞争的运行表现。加入门诊和体检后，排入分钟由 1 344 765 增至 1 897 045，增加 41.07%；人员在岗容量利用率由 54.53% 升至 76.93%，住院等待 P95 由 23.67 小时增至 25.58 小时。固定需求分母下的多项目跨室率由 58.16% 降至 30.63%，原因是共享容量收紧后，进入完整完成集合的住院事件组合发生变化；该指标以评价期全部多项目住院需求为分母，并非已完成患者条件率，因此必须与住院完成率和未完成原因共同解释，不能单独视为跨室体验改善。

5.3.5 推荐排程的运行形态

推荐联合方案共导出 159 580 条已排任务，覆盖 100 296 个完整排入事件；患者级回滚机制不保留部分排入事件。评价期内进入完成分子的事件为 99 998 个，其中住院 45 861 个、背景 54 137 个；其余已排事件属于构造连续日历所需的预热状态。图 7 显示日级率受星期结构、日需求规模和项目组合影响，14 日滚动后仍有明显波动，说明日常运行需要同时监控总体完成率与逐日压力。

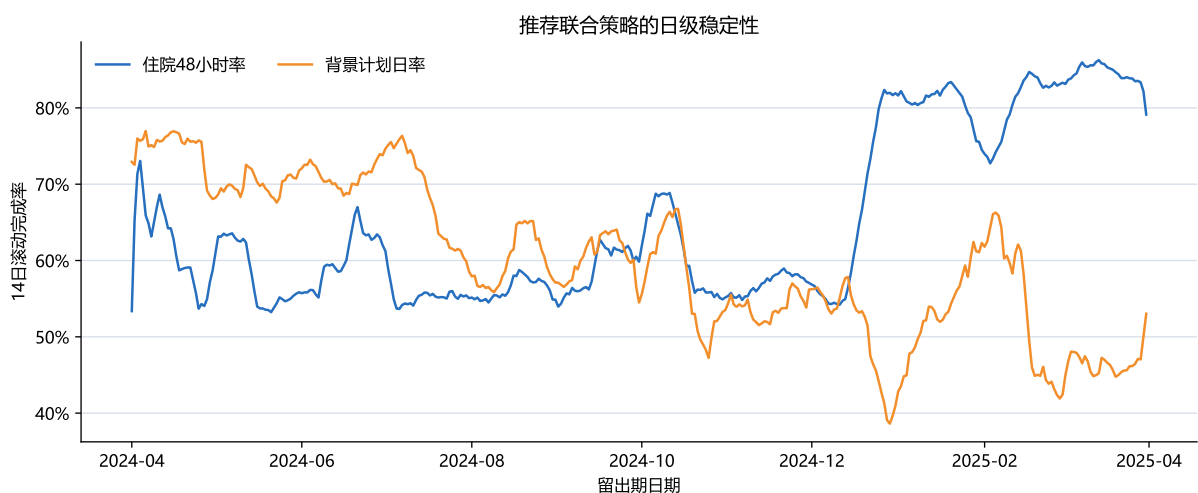


图 7 推荐联合策略的日级稳定性

图 8 给出总排入分钟较高的代表日设备甘特图。每个条块对应一个患者项目，排程使用题给 22 台设备、上午和下午工作块及项目设备兼容关系；午休时段保持空闲，床旁 7 号机和专项诊室只承接各自可行任务。

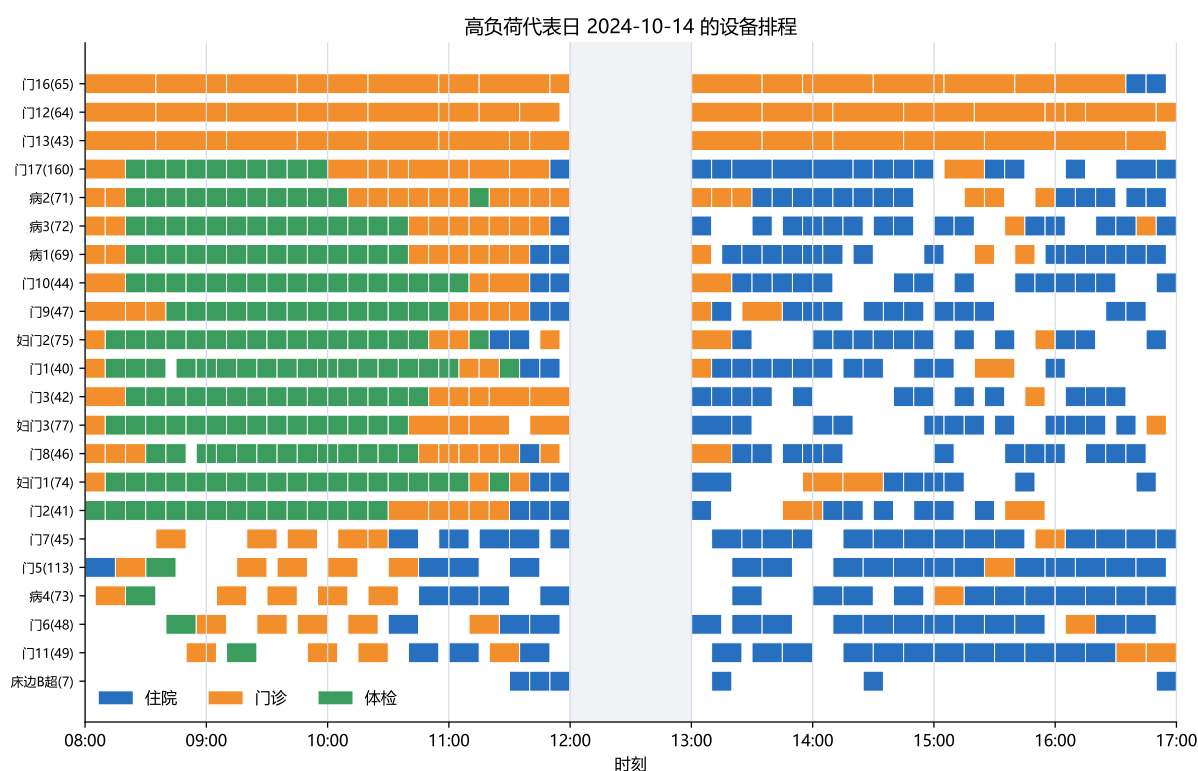


图 8 高负荷代表日的设备排程

六、模型的检验

6.1 需求预测模型检验

需求预测按时间划分训练期与留出期。2019 年 3 月 1 日至 2024 年 3 月 31 日用于估计日历趋势、星期效应与模型参数，2024 年 4 月 1 日至 2025 年 3 月 31 日仅用于评价。滚动 8 周模型在每个预测日只读取此前数据。采用 WAPE 和 sMAPE 比较 7 日季节朴素、过去 8 周同星期中位数与日历趋势对数岭回归，结果见表 5。住院和门诊由 8 周同星期

中位数取得最低 WAPE，分别为 16.04% 和 14.04%；体检受团检批次影响，最低 WAPE 仍为 69.83%。因此，住院和门诊可由历史星期节律滚动预测，体检配置需要未来批次计划或高分位压力情景。该现象与医院门诊量中常见的趋势和周内效应一致 [9, 10]。

模型参数的时间边界另行核对：峰平阈值、项目目录、医生容量和设备稀缺权重只从训练期提取；留出期不参与参数选择。报告间隔仅作项目族相对负荷代理，排程时长仍使用题给区间。由此，表 5 中的误差反映向后续日期外推的表现，而不是样本内拟合程度。

6.2 调度模型有效性检验

为检验调度对象的构造口径，重新核对事件键、项目数与设备集合。精确重复明细只计一次，同一患者同一开单时刻的不同项目仍归入同一事件；源系统显式连接的项目均按边界拆分。设备集合逐项对应题目表 1，无法确认的项目保持空集合；床旁任务按式 (19) 取项目能力与 7 号机地点能力的交集。

问题二在患者顺序、候选项目序列、设备破同序、准备条件和转运规则均一致的条件下构造启发式对照。为使精确模型能够在有限时间内求解，本文从留出期数据按负荷、项目结构和设备约束抽取并额外构造 6 个代表性子问题；这些子问题是算法验证样本，并非赛题规定的运行情景。CP-SAT 在每个子问题中最大化完整完成事件数，表 14 同时报告精确完成数、最好界和求解状态；只有求解器状态为 OPTIMAL 时，精确完成数才作为该子问题的已证最优值。

表 14 问题二构造算法与 CP-SAT 精确子问题对照

验证子问题	事件数	构造解	精确可行值	最好界	状态	至界差距
普通工作日	20	17	17	17	OPTIMAL	0.00%
高峰工作日	100	90	90	90	OPTIMAL	0.00%
周末	40	37	37	37	OPTIMAL	0.00%
多项目高峰	40	36	36	36	OPTIMAL	0.00%
床旁高峰	20	2	2	2	OPTIMAL	0.00%
产科高峰	40	36	36	36	OPTIMAL	0.00%

6/6 个子问题包含 20–100 个事件，均由求解器证明为 OPTIMAL；构造算法的完整完成数与精确模型可行值逐个相同，最大保守至界差距为 0.00%。这说明本文构造算法在所抽取的普通、高峰、周末、多项目、床旁和产科局部样本中达到已证最优，但该结论不外推为全年大规模问题的最优性证明。

问题三的精确子问题同时包含住院、门诊、体检、背景最低完成数以及全部资源和准备约束。本文从留出期按需求结构和资源竞争程度抽取并额外构造 8 类代表性子问题，覆盖普通联合需求、背景高峰、住院高峰、多项目、床旁、专项设备和高竞争样本；它们用于验证算法，不是赛题预设情景。每个子问题按自身背景事件数施加与构造算法相同的局部 $\alpha = 0.45$ 约束，仅用于同规模解法对照；该缩小模型既不替代全年全局约束，也不表示正式模型设置了逐日配额。

表 15 问题三联合构造算法与 CP-SAT 精确子问题对照

验证子问题	α	事件数	背景 H/E	H 住院	E 住院值 最好界	状态	求解器 间隙	H 至界 差距
普通联合	0.45	20	12/7	4	4/4	OPTIMAL	0.00%	0.00%
背景高峰	0.45	40	30/14	6	6/6	OPTIMAL	0.00%	0.00%
住院高峰	0.45	60	24/11	32	32/32	OPTIMAL	0.00%	0.00%
多项目高峰	0.45	40	19/11	14	14/14	OPTIMAL	0.00%	0.00%
床旁压力	0.45	20	8/6	0	0/0	OPTIMAL	0.00%	0.00%
产科压力	0.45	40	23/11	15	15/15	OPTIMAL	0.00%	0.00%
设备竞争	0.45	60	34/17	23	23/23	OPTIMAL	0.00%	0.00%
推荐点典型窗	0.45	100	60/27	33	33/33	OPTIMAL	0.00%	0.00%

表 15 中 H、E 分别表示构造算法与精确模型，背景列按 H/E 并列；“E 住院值/最好界”给出时限内精确模型的住院可行值 E 与最好界 U 。两类间隙均针对住院完成数目标：

$$G_{\text{solver}} = \frac{U - E}{\max\{|U|, 1\}}, \quad (56)$$

$$G_{\text{H,bd}} = \frac{U - H}{\max\{|U|, 1\}}. \quad (57)$$

当状态为 OPTIMAL 时， $E = U$ ， $G_{\text{H,bd}}$ 就是构造解相对已证最优值的差距；当状态为 FEASIBLE 时，只报告可行值、最好界和两类保守间隙，不将可行值或最好界表述为最优解。8/8 个代表性子问题均为 OPTIMAL，构造解的住院完成数逐个等于已证最优值，最大保守至界差距为 0.00%。但这 8 个子问题的住院优先基线都已满足局部背景下限，没有触发全局缺口重构，因此上述结果只为基线分支提供局部最优证据，不能证明修复分支或全年联合问题达到全局最优。

为补充修复分支的实际路径证据，另按留出期题给上界工时选择 2025 年 2 月 17 日作为真实峰值日。该日共 805 个事件，其中住院 305 个、门诊 407 个、体检 93 个，上界工时合计 20 375 分钟；局部背景事件为 500 个， $\alpha = 0.45$ 对应最低完成数 225。住院优先基线仅完成背景 213 个、住院 259 个，背景缺口为 12，因而自然触发全局重构；稀缺能力优先修复完成背景 258 个、住院 259 个并被选中，14 项独立检查的违规数均为 0。该审计只证明真实数据中修复分支能够被自然触发并产生可行排程，不是修复分支的精确最优性证明。

为避免求解与验证共享同一段可行性判断，另从排程结果重建任务时长、释放与截止、设备能力、准备条件、设备互斥、患者互斥、跨室转运和医生并发约束。结果见表 16。问题二 117 843 条排入任务与问题三 159 580 条排入任务的各组违规数均为 0，观察到的最大医生并发为 20，未超过主情景容量。

表 16 问题二、三导出排程的独立可行性验证

核验组	P2 违规	P3 违规	结论
键、输入与时长	0	0	通过
设备能力与床旁	0	0	通过
时间窗与准备	0	0	通过
互斥与转运	0	0	通过
医生容量	0	0	通过
结果聚合	0	0	通过

现实校准按 C0–C7 逐层加入标准工时、工作时段、医生并发、项目设备能力、项目级准备和三类患者竞争，见表 17 与图 9。历史开单至报告率为 83.77%，其时间语义与排程完成率不同，仅作为现实参照。仅加入题给工时后完成率为 100%，较历史口径高 16.23 个百分点；标准班次和训练期医生并发加入后仍为 100%。项目设备能力使完成率降至 85.51%，相邻下降 14.49 个百分点；项目级准备进一步使其降至 85.30%，相邻下降 0.20 个百分点。门诊、体检按 $\alpha = 0.45$ 进入共享日历后，住院率降至 64.80%，相邻下降 20.50 个百分点，说明专项设备集合和三类需求竞争是两项主要约束。C7 仍为 64.80%、相邻变化 0.00 个百分点，因为该层只记录正式构造方法，没有再次改变可行域。

表 17 历史现实校准与约束增量

校准层	完成数/分母	完成率	较上一层/百分点
历史开单至报告	59,283/70,771	83.77%	—
仅题给时长	70,771/70,771	100.00%	+16.23
加入标准班次	70,771/70,771	100.00%	+0.00
加入医生并发	70,771/70,771	100.00%	+0.00
加入项目设备能力	60,513/70,771	85.51%	-14.49
加入项目级准备	60,368/70,771	85.30%	-0.20
加入门诊/体检竞争	45,861/70,771	64.80%	-20.50
正式构造策略	45,861/70,771	64.80%	+0.00

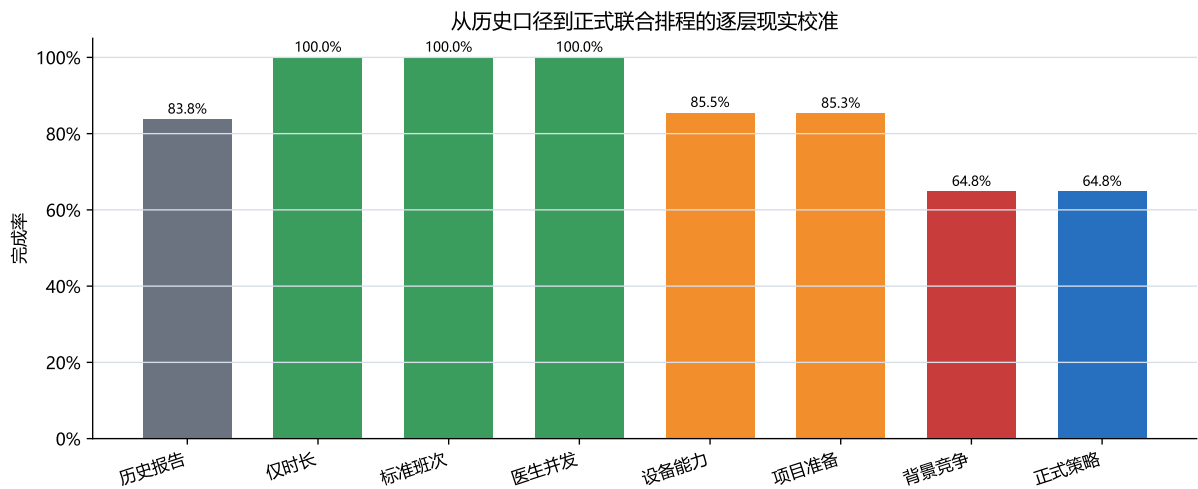


图 9 从历史口径到正式联合排程的逐层现实校准

6.3 灵敏度分析

保持评价期和推荐下限 $\alpha = 0.45$ 不变，分别改变医生容量、设备匹配边界、准备粒度、憋尿时间和转运时间，结果见表 18 和图 10。每个情景只改变一个主因素，以便区分参数影响。表中同时给出 ε 约束是否满足及背景缺口；只有达到背景下限的情景才可作为同一政策约束下的可行方案比较，未达标情景只反映当前构造算法在该参数下的搜索结果。

表 18 推荐联合方案的单因素敏感性

情景	住院率	背景率	P90/h	ε 满足	背景缺口	排入分钟
主设定：项目级准备	64.80%	56.37%	21.83	是	0	1,897,045
事件级准备	63.02%	56.30%	24.83	是	0	1,853,985
医生容量下四分位	63.33%	54.38%	26.67	是	0	1,819,755
仅直接/语义匹配	65.26%	47.98%	21.33	是	0	1,842,275
憋尿 45 分钟	64.84%	58.59%	29.67	是	0	2,077,830
憋尿 90 分钟	64.74%	55.96%	21.92	是	0	1,892,615
跨室 5 分钟	64.85%	58.11%	29.17	是	0	2,056,460
跨室 15 分钟	64.82%	56.35%	21.92	是	0	1,898,420

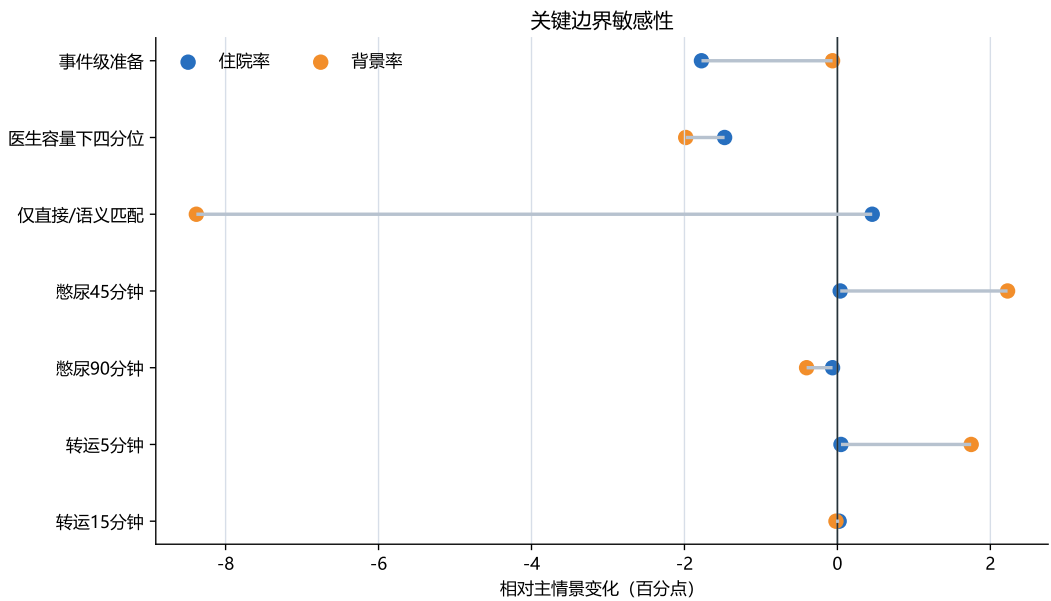


图 10 主要参数变化相对主情景的完成率差异

八个单因素情景均满足 ε 约束，达标数为 8/8，背景缺口均为 0。医生容量降至训练期下四分位时，住院率和背景率分别为 63.33% 和 54.38%，等待 P90 增至 26.67 小时，表明人员在岗容量下降会同时压低两类服务。只保留直接与明确语义匹配时，住院率为 65.26%，背景率降至 47.98%；住院率略升并不表示资质收紧改善系统，而是候选设备集合变化后，两类需求的竞争组合发生了改变。

事件级准备的住院率为 63.02%，低于项目级准备的 64.80%，背景率二者接近，分别为 56.30% 和 56.37%；其等待 P90 由 21.83 小时升至 24.83 小时，支持将准备条件落实到对应项目。憋尿 45/90 分钟与转运 5/15 分钟四个边界情景中，两类完成率相对主情景的最大变化为 2.22 个百分点，出现在憋尿 45 分钟的背景率；同时该情景等待 P90 达

到 29.67 小时。因此这些参数没有破坏 $\alpha = 0.45$ 下的可行性，但可能改变构造分支和等待尾部，不能只凭总体完成率判断其影响。

6.4 稳健性分析

复杂病例按题设比例抽取，采用五个固定随机种子重复运行。五次运行均满足 ε 约束，达标数为 5/5，背景缺口均为 0。住院率样本标准差仅为 0.0114 个百分点，取值为 64.7935%–64.8189%，说明住院完成率对复杂病例具体落点较稳定；背景率样本标准差为 0.9495 个百分点，取值为 56.3209%–58.4584%。等待 P90 范围为 21.83–29.42 小时，其中一个种子切换到稀缺能力优先的背景重构分支，产生更高的等待尾部。由此可认为 $\alpha = 0.45$ 下的约束可行性具有多种子稳定性，但背景服务量和等待尾部仍对离散构造路径敏感。

表 19 复杂病例多随机种子稳健性

指标	均值	标准差	最小值	最大值
住院 48 小时率	64.8039%	0.0114 个百分点	64.7935%	64.8189%
背景服务率	56.7601%	0.9495 个百分点	56.3209%	58.4584%
等待 P90/小时	23.38	3.37	21.83	29.42
排入分钟	1,932,254	81,073	1,895,420	2,077,280

需求状态方面，问题一用训练期分位数识别平峰、高峰与压力状态，留出期超过训练高峰阈值的日期占 13.15%，超过压力阈值的日期占 7.95%。问题二与问题三的精确对照样本分别纳入高负荷、多项目、床旁和专项设备约束；日级完成率与 14 日滚动率同时保留，用于识别总体率掩盖的短期波动。预测留出、精确子问题、独立可行性复核、单因素敏感性和多种子检验从不同角度限定了模型结果的适用范围。

七、 模型的评价

7.1 模型的优点

- (1) 评价对象与题意一致。模型以患者全部项目完整完成为计数单元，未排项目和超过截止的事件均保留在评价分母中。问题二只处理住院患者，问题三再加入门诊和体检需求，两种场景的完成率具有明确的资源边界。
- (2) 设备与准备条件具有现实对应。22 台设备按题目表 1 逐项目建立兼容集合，复合项目按组成项目确定共同能力；床旁位置与项目能力取交集。空腹、憋尿、工作时段和跨室转运均落实到具体项目，避免把不同检查路径视为同质任务。分时段、分项目组预约在实际超声流程中也具有改善候检秩序的应用基础 [4, 3]。
- (3) 统计参数和调度决策衔接完整。需求阈值、项目目录与医生容量由训练期确定，留出期用于评价；题给时长进入主排程，历史报告间隔只辅助判断相对负荷。问题一输出的工时、兼容集合和时隙容量直接构成问题二、三模型参数，并与超声工作量和设备质量管理指标的业务口径相衔接 [17]。
- (4) 大规模求解保持患者级完整性。构造算法枚举患者内部候选序列，优先搜索共同设备，并在全部项目可行后整体提交。该机制与完整事件目标一致，也保留了设备互斥、医生并发、准备和转运等约束。从留出期抽取并额外构造的代表性子问题用于精确对照，最终排程再通过独立计算重建约束。精确对照只覆盖未触发修复的基线分支，真实峰值日审计则只检查修复的自然触发与约束可行性，两类证据的边界彼此分开。
- (5) 多目标结果便于政策选择。问题三把背景服务水平写成全评价期下限，以 ε 约束生成非支配点。医院可根据门诊、体检政策下限选择对应方案，并同时观察住院完成率、等待时间和资源利用率，而不必预先给两个目标设置难以解释的权重。

7.2 模型的不足

(1) 报告完成时刻采用代理。附件没有扫查结束、报告书写、审核和提交时刻，模型以最后项目结束近似报告完成。因此，排程完成率适合评价预约与扫查资源，尚不能单独刻画报告环节的真实延迟。

(2) 背景计划日来自回顾性信息。门诊、体检报告日期不是原始预约日期，现有压力场景未包含爽约、迟到和改约行为。背景完成率表示标准班次的计划日承载能力，部署时仍需接入真实预约和到达数据。

(3) 医生能力只刻画总并发。题目给出医生总量，但历史医生与未来 33 名医生之间没有实名映射。当前模型按星期与 5 分钟槽估计共享并发容量，尚未描述每名医生的专项资质、班次、请假和设备偏好，专项供给可能因此被高估。

(4) 部分服务参数仍为区间近似。多数项目采用题给时长区间，模型尚未利用同部位多项目可能存在的准备共享、扫查协同和报告共享。憋尿与跨室时间通过敏感性给出边界，但患者个体差异仍未进入主模型。

(5) 全年求解采用构造式算法。全年问题规模较大，主模型采用患者级构造与修复控制计算量。8 个代表性子问题只为住院优先基线提供局部最优证据，真实峰值日修复审计也不证明修复分支最优；当高背景下限未被当前候选搜索满足时，不能据此认定数学模型不可行。对于极端高负荷和长程资源竞争，仍需用滚动 CP-SAT 或局部重优化扩大搜索。

(6) 公平性尚未成为主目标。当前词典序首先提高完整事件数，项目少或设备选择较多的患者可能更容易排入。模型虽保留全部患者，但尚未对科室、严重程度和多项目患者设置最低服务率。推荐结果适用于题给设备、标准日班和当前需求结构；资源数量、资质集合或工作时段改变后需要重新求解。

八、模型的改进与推广

8.1 模型的改进

(1) 补充真实报告流程。当前以最后检查结束近似报告完成。若获得预约、到达、扫查开始与结束、报告提交和审核时刻，可把患者流程拆为开单、预约、到达、扫查、报告和审核六个环节，并估计各环节时长分布。住院目标可改写为机会约束

$$\Pr\{C_i^{\text{report}} \leq p_i + 48 \text{ h}\} \geq 1 - \epsilon_i, \quad (58)$$

从而把预约拥堵与报告延迟分开，并按项目风险设置缓冲。

(2) 加入实名医生资质。获得未来医生集合 $\mathcal{A}$ 、医生项目资质 h_{aj} 和实名班次 B_{av} 后，引入医生指派变量 u_{ijat} ，表示项目 (i, j) 在时隙 t 开始且由医生 a 执行。指派关系满足

$$u_{ijat} \leq h_{aj}, \quad (59)$$

$$\sum_{i,j} \sum_{t:t \leq v < t + \ell_{ij}} u_{ijat} \leq B_{av}, \quad (60)$$

$$\sum_a u_{ijat} = \sum_r x_{ijrt}. \quad (61)$$

该扩展把总并发容量细化为设备、医生与项目三维匹配，还可增加医生换房惩罚以减少人员走动。

(3) 采用滚动时域重排。真实运行中，患者到达、医嘱变化和设备故障会持续更新。每个决策时刻 k 重建

$$\mathcal{I}^{(k)} = \mathcal{I}_{\text{unfinished}}^{(k)} \cup \mathcal{I}_{\text{booked}}^{(k:k+H)} \cup \mathcal{I}_{\text{new}}^{(k)}, \quad (62)$$

固定已开始及临近开始的任务，仅重排窗口 H 内其余事件。目标中加入改约成本

$$\lambda_{\text{chg}} \sum_i \left| s_i^{(k)} - s_i^{(k-1)} \right|, \quad (63)$$

以平衡完成率提升与患者通知成本。

(4) 增加公平服务约束。对科室、严重程度或多项目患者组 g 定义完成率 R_g ，可设置

$$R_g \geq \rho_g, \quad (64)$$

$$\max_g R_g - \min_g R_g \leq \Delta. \quad (65)$$

实施时需要可靠的严重程度标签和政策阈值。医院也可采用分层词典序目标，先满足急危重与最低服务标准，再提高总体完成数和等待表现。

(5) 联合评估容量扩展。当设备增购、工作时段延长和医生增班成为决策变量时，可建立上层容量配置、下层患者调度的双层模型：

$$\min \sum_r c_r^{\text{device}} u_r + \sum_t c_t^{\text{overtime}} v_t + \kappa(1 - R_I). \quad (66)$$

其中 u_r 表示新增某类设备， v_t 表示扩展时段。校准结果表明设备专项能力和三类需求竞争需要共同评估，容量建设应比较床旁、产科、心脏等专项设备与医生班次的边际收益。

8.2 模型的推广

本文模型的基本结构是“患者—项目—设备—人员—时间窗”。迁移到 CT 或 MRI 预约时，项目设备集合对应扫描协议与增强资质，准备条件需要加入造影剂、禁食和安全筛查；迁移到内镜时，应重新估计清洗、麻醉与恢复床容量；迁移到放疗时，项目序列扩展为定位、计划和分次治疗；迁移到手术室时，设备资源改为手术间，人员资源扩展为术者、麻醉和护理团队。

推广时可保留患者完整流程目标、异质资源兼容集合、共享人员容量和时间窗约束，重新估计项目时长、资源资质、工作时段、准备与转运参数。对存在急诊插入、设备故障或爽约的场景，还需加入随机到达和滚动重排，并重新完成预测留出、精确子问题和敏感性检验。这样才能使数学结构保持一致，同时避免直接沿用本题的设备数量和完成率。

九、 参考文献

- [1] GUPTA D, DENTON B. Appointment scheduling in health care: Challenges and opportunities[J]. IIE Transactions, 2008, 40(9): 800–819. DOI: <https://doi.org/10.1080/07408170802165880>.
- [2] AHMADI-JAVID A, JALALI Z, KLASSEN K J. Outpatient appointment systems in healthcare: A review of optimization studies[J]. European Journal of Operational Research, 2017, 258(1): 3–34. DOI: <https://doi.org/10.1016/j.ejor.2016.06.064>.
- [3] 国家卫生健康委员会办公厅. 关于进一步完善预约诊疗制度加强智慧医院建设的通知: 国卫办医函(2020)405号 [EB/OL]. (2020-05-21)[2026-08-24]. <https://www.nhc.gov.cn/yzygj/c100068/202005/43b2d23ff48448ffae96700bc6eaccd7.shtml>.
- [4] 陈娟, 李志会, 赵浩, 等. 分时段分项目组多途径预约模式在妇幼专科医院超声检查中的应用 [J]. 中国妇幼卫生杂志, 2023, 14(6): 77–80. DOI: <https://doi.org/10.19757/j.cnki.issn1674-7763.2023.06.014>.
- [5] MARYNISSEN J, DEMEULEMEESTER E. Literature review on multi-appointment scheduling problems in hospitals[J]. European Journal of Operational Research, 2019, 272(2): 407–419. DOI: <https://doi.org/10.1016/j.ejor.2018.03.001>.
- [6] APERGI L A, BARAS J S, GOLDEN B L, et al. An optimization model for multi-appointment scheduling in an outpatient cardiology setting[J]. Operations Research for Health Care, 2020, 26: 100267. DOI: <https://doi.org/10.1016/j.orhc.2020.100267>.
- [7] 王珊珊, 李金林, 彭春, 等. 不确定服务时间下分布式鲁棒门诊预约调度和排程 [J]. 系统工程学报, 2019, 34(4): 566–576.
- [8] 华容区人民医院. 超声科流程及注意事项 [EB/OL]. (2025-08-20)[2026-08-24]. http://www.hbhr.gov.cn/hrqxxgk/xxgkml/ggqsydwxxgk/wsjk/hrqrmyy/202508/t20250821_720866.html.
- [9] 朱顺痣, 王大寒, 何亚男, 等. 基于时间序列模型的医院门诊量分析与预测 [J]. 中国科学技术大学学报, 2015, 45(10): 795–803. DOI: <https://doi.org/10.3969/j.issn.0253-2778.2015.10.001>.
- [10] 向前, 陈平雁. 预测医院门诊量的 ARIMA 模型构建及应用 [J]. 南方医科大学学报, 2009, 29(5): 1076–1078. PMID: 19460744.
- [11] LUO L, LUO L, ZHANG X, et al. Hospital daily outpatient visits forecasting using a combinatorial model based on ARIMA and SES models[J]. BMC Health Services Research, 2017, 17: 469. DOI: <https://doi.org/10.1186/s12913-017-2407-9>.
- [12] LAI C H, LU Y J, CHEN P S. Using data-driven techniques to predict outpatient ultrasound examination time for the multi-clinic outpatient appointment scheduling problem[J]. Communications in Statistics-Simulation and Computation, 2025, 54(9): 3358–3376. DOI: <https://doi.org/10.1080/03610918.2024.2349168>.

- [13] PATRICK J, PUTERMAN M L, QUEYRANNE M. Dynamic multipriority patient scheduling for a diagnostic resource[J]. *Operations Research*, 2008, 56(6): 1507–1525. DOI: <https://doi.org/10.1287/opre.1080.0590>.
- [14] PATRICK J, PUTERMAN M L. Improving resource utilization for diagnostic services through flexible inpatient scheduling: A method for improving resource utilization[J]. *Journal of the Operational Research Society*, 2007, 58(2): 235–245. DOI: <https://doi.org/10.1057/palgrave.jors.2602242>.
- [15] CHEN P S, CHEN G Y H, LIU L W, et al. Using simulation optimization to solve patient appointment scheduling and examination room assignment problems for patients undergoing ultrasound examination[J]. *Healthcare*, 2022, 10(1): 164. DOI: <https://doi.org/10.3390/healthcare10010164>.
- [16] 乔岩, 冉伦, 李金林, 等. 基于两阶段随机规划的远程会诊预约调度问题研究 [J]. *中国管理科学*, 2024, 32(1): 86–93. DOI: <https://doi.org/10.16381/j.cnki.isn1003-207x.2020.1989>.
- [17] 国家卫生健康委员会办公厅. 关于印发超声诊断等 5 个专业医疗质量控制指标 (2022 年版) 的通知: 国卫办医函 (2022) 161 号 [EB/OL]. (2022-05-27)[2026-08-24]. <https://www.nhc.gov.cn/yzygj/c100068/202205/164e6556f6d34c33a3344fec9a2c5074.shtml>.

附录 A 支撑材料文件说明

计算所用程序和结果数据保存在 **supporting_materials** 目录。限于篇幅，本附录选录与各模型直接对应的核心算法，主要文件见表 20。

表 20 主要程序及功能

文件	功能
<code>code/analyze_p1.py</code>	构造申请事件，分析需求、项目工时、科室结构与医生容量
<code>code/build_unified_inputs.py</code>	生成统一事件表、项目表和项目设备兼容集合
<code>code/run_unified_scheduler.py</code>	患者级可行性内核及问题二纯住院调度
<code>code/run_pre_freeze_scenarios.py</code>	问题二候选策略与问题三背景下限扫描
<code>code/benchmark_exact_scheduler.py</code>	问题二 CP-SAT 精确子问题对照
<code>code/benchmark_exact_p3_scheduler.py</code>	问题三联合调度 CP-SAT 精确子问题对照
<code>code/verify_unified_schedule.py</code>	从导出排程独立重建并检查全部约束
<code>results/p1/</code>	问题一需求、工时、峰平阈值与预测检验结果
<code>results/final_frozen/</code>	问题二、问题三及敏感性结果的机器可读文件

计算顺序与正文一致：先完成数据预处理和问题一分析，再构建统一调度输入并求解问题二、三，随后计算精确子问题、校准和敏感性情景，最后独立检验排程可行性。各程序从题目原始附件或上一步结果读取数据，完整字段说明保存在同目录说明文档中。

附录 B 数据预处理程序

数据预处理先由 `analyze_p1.py` 中的 `expand_embedded_project_lists` 按源系统明确的] [边界拆分项目，再由 `build_unified_inputs.py` 中的 `load_source` 按“来源、患者编号、精确开单时刻”聚合事件、删除精确重复明细并生成稳定事件编号。下面直接摘录实际源码；函数后续的项目表字段整理保留在完整文件中。

Listing 1 显式复合项目拆分的实际源码

```

218 def expand_embedded_project_lists(frame: pd.DataFrame) -> pd.DataFrame:
219     """Split the source system's explicit ``[`` multi-item encoding.
220
221     A single order field can contain several separately billed examinations,
222     encoded as ``[item,modality][item,modality]``. Only the bracket boundary
223     is used as a separator; commas and Chinese enumeration marks inside an
224     examination name are retained. This preserves compound anatomy names
225     while making the scheduling task count agree with the source semantics.
226     """
227     expanded = frame.copy()
228     expanded["ordered_project_original"] = expanded["ordered_project"]
229     expanded["ordered_project"] = clean_text(expanded["ordered_project"]).str.split(
230         r"\]\s*\[", regex=True
231     )
232     expanded = expanded.explode("ordered_project", ignore_index=True)
233     expanded["ordered_project"] = (
234         clean_text(expanded["ordered_project"])
235         .str.replace(r"^\[\]\,|\[\[\]\,]+$", "", regex=True)
236     )
237     expanded["source_component_index"] = expanded.groupby("source_row").cumcount() + 1
238     return expanded

```

Listing 2 事件聚合与去重的实际源码

```

141 def load_source(source: str, filename: str) -> pd.DataFrame:
142     frame = p1.load_detail(RAW / filename, source)
143     frame["item_bedside"] = as_bool(frame["床旁标记"])
144     frame["item_fasting"] = as_bool(frame["空腹上午标记"])
145     frame["item_bladder"] = as_bool(frame["充盈膀胱标记"])
146     frame["patient_id"] = clean_id(frame["patient_id"])
147     missing = frame["patient_id"].isna()
148     frame.loc[missing, "patient_id"] = "MISSING_" + frame.loc[missing, "source_row"].
    astype(str)
149     frame["report_date"] = frame["report_dt"].dt.normalize()
150
151     frame = frame.sort_values("source_row", kind="stable")
152
153     duplicate_columns = [
154         "patient_id",
155         "order_dt",
156         "ordered_project",
157         "report_dt",
158         "exam_doctor_id",
159         "machine_id",
160     ]
161     frame["is_exact_duplicate_extra"] = frame.duplicated(duplicate_columns, keep="
    first")
162     frame = frame[~frame["is_exact_duplicate_extra"]].copy()
163
164     event_keys = ["source", "patient_id", "order_dt"]
165     event_flags = (
166         frame.groupby(event_keys, dropna=False)
167         .agg(
168             bedside=("床旁标记", "max"),
169             fasting=("空腹上午标记", "max"),
170             bladder=("充盈膀胱标记", "max"),
171             planned_date=("report_date", mode_date),
172         )
173         .reset_index()
174     )
175     frame = frame.merge(event_flags, on=event_keys, how="left", validate="many_to_one"
    )
176     frame = frame[~frame["non_service_item"] & frame["project_norm"].notna()].copy()
177     frame = frame.drop_duplicates(event_keys + ["project_norm"], keep="first")

```

Listing 3 模拟边界与稳定事件编号的实际源码

```

179 # Construct the complete application event before applying the simulation
180 # boundary. Filtering detail rows first can truncate a multi-project event
181 # whose reports fall on different dates and can change its modal plan date.
182 if source == "住院":
183     keep = frame["order_dt"].between(
184         SIMULATION_START, HOLDOUT_END_EXCLUSIVE, inclusive="left"
185     )
186 else:
187     keep = frame["planned_date"].between(
188         SIMULATION_START, HOLDOUT_END_EXCLUSIVE, inclusive="left"
189     )
190 frame = frame[keep].copy()
191
192 keys = (
193     frame[event_keys]
194     .drop_duplicates()
195     .sort_values(["order_dt", "patient_id"], kind="stable")
196     .reset_index(drop=True)
197 )
198 code = SOURCE_CODES[source]

```

```

199 keys["event_id"] = [f"{code}2024-{index:06d}" for index in range(1, len(keys) + 1)
200 ]
201 frame = frame.merge(keys, on=event_keys, how="left", validate="many_to_one")
202 return frame

```

附录 C 问题一程序

问题一按来源构造完整日序列，严格以 2024 年 4 月 1 日为时间分界，在训练期估计候选预测器并在留出期计算误差。对数岭回归由线性方程直接求解，峰平阈值只读取训练子集。以下均为 `analyze_p1.py` 的实际代码。

Listing 4 时间留出预测与误差计算的实际源码

```

533 def chronological_validation(daily: pd.DataFrame) -> pd.DataFrame:
534     results: list[dict[str, Any]] = []
535     series_frames: list[tuple[str, str, pd.DataFrame]] = []
536     for (source, basis), group in daily.groupby(["source", "basis"]):
537         series_frames.append((source, basis, group.sort_values("date").copy()))
538     for basis, group in daily.groupby("basis"):
539         combined = (
540             group.groupby("date", as_index=False)["episode_count"].sum().sort_values("
date")
541         )
542         series_frames.append(("三类合计", basis, combined))
543
544     for source, basis, frame in series_frames:
545         frame = frame.sort_values("date").reset_index(drop=True)
546         y = frame["episode_count"].to_numpy(dtype=float)
547         dates = frame["date"]
548         train_mask = dates.lt(VALIDATION_START).to_numpy()
549         test_mask = dates.between(VALIDATION_START, VALIDATION_END).to_numpy()
550         actual = y[test_mask]
551
552         predictions: dict[str, np.ndarray] = {}
553         predictions["7日季节朴素"] = pd.Series(y).shift(7).to_numpy()[test_mask]
554
555         same_weekday_median = np.full(len(frame), np.nan)
556         for index in np.flatnonzero(test_mask):
557             history_index = [index - 7 * lag for lag in range(1, 9) if index - 7 * lag
>= 0]
558             same_weekday_median[index] = np.median(y[history_index])
559         predictions["过去8周同星期中位数"] = same_weekday_median[test_mask]
560
561         design_train = calendar_design(dates[train_mask], START_DATE)
562         response_train = np.log1p(y[train_mask])
563         ridge = 1e-4 * np.eye(design_train.shape[1])
564         ridge[0, 0] = 0
565         coefficient = np.linalg.solve(
566             design_train.T @ design_train + ridge,
567             design_train.T @ response_train,
568         )
569         design_test = calendar_design(dates[test_mask], START_DATE)
570         predictions["日历趋势对数岭回归"] = np.maximum(np.expm1(design_test @
coefficient), 0)
571
572     for model, predicted in predictions.items():
573         finite = np.isfinite(predicted) & np.isfinite(actual)
574         metrics = metric_row(actual[finite], predicted[finite])
575         results.append(
576             {
577                 "source": source,
578                 "basis": basis,
579                 "model": model,

```

```

580         "train_start": str(dates[train_mask].min().date()),
581         "train_end": str(dates[train_mask].max().date()),
582         "validation_start": str(dates[test_mask].min().date()),
583         "validation_end": str(dates[test_mask].max().date()),
584         "validation_days": int(finite.sum()),
585         **metrics,
586     }
587 )
588 result = pd.DataFrame(results)
589 result["best_WAPE_within_series"] = result.groupby(["source", "basis"])["WAPE"].
transform("min").eq(result["WAPE"])
590 return result

```

Listing 5 训练期峰平分位数的实际源码

```

593 def peak_flat_summary(daily: pd.DataFrame) -> pd.DataFrame:
594     frames: list[tuple[str, str, pd.DataFrame]] = []
595     for (source, basis), group in daily.groupby(["source", "basis"]):
596         frames.append((source, basis, group.copy()))
597     for basis, group in daily.groupby("basis"):
598         frames.append(
599             (
600                 "三类合计",
601                 basis,
602                 group.groupby("date", as_index=False)["episode_count"].sum(),
603             )
604         )
605     rows: list[dict[str, Any]] = []
606     weekdays = ["周一", "周二", "周三", "周四", "周五", "周六", "周日"]
607     for source, basis, frame in frames:
608         training = frame[
609             frame["date"].between(START_DATE, VALIDATION_START - pd.Timedelta(days=1))
610         ].copy()
611         holdout = frame[frame["date"].between(VALIDATION_START, VALIDATION_END)].copy()
612         training["weekday"] = training["date"].dt.dayofweek
613         training["month"] = training["date"].dt.month
614         values = training["episode_count"]
615         q45, q50, q55, q90, q95 = values.quantile([0.45, 0.5, 0.55, 0.9, 0.95])

```

附录 D 问题二程序

问题二由 `task_order_candidates` 生成四类项目顺序, `tentative_plan` 逐项预留日历; 任一任务失败时逆序释放此前位置, 全部成功时保留预留结果。 `plan_event` 先尝试共同设备, 再放开跨室安排。下面直接摘录 `run_unified_scheduler.py` 中的实现。

Listing 6 候选项目序列与失败回滚的实际源码

```

484 def task_order_candidates(event_tasks: list[Task]) -> list[list[Task]]:
485     keys = [
486         lambda task: (not task.fasting, len(task.compatible_rooms), -task.
duration_slots, task.item_index),
487         lambda task: (not task.fasting, -task.duration_slots, len(task.
compatible_rooms), task.item_index),
488         lambda task: (len(task.compatible_rooms), not task.fasting, task.item_index),
489         lambda task: (task.item_index,),
490     ]
491     candidates: list[list[Task]] = []
492     seen: set[tuple[int, ...]] = set()
493     for key in keys:
494         ordered = sorted(event_tasks, key=key)

```

```

495     signature = tuple(task.item_index for task in ordered)
496     if signature not in seen:
497         candidates.append(ordered)
498         seen.add(signature)
499     return candidates
500
501
502 def tentative_plan(
503     calendar: WorkCalendar,
504     ordered_tasks: list[Task],
505     release: pd.Timestamp,
506     latest_end: pd.Timestamp,
507     room_override: str | None,
508 ) -> list[PlanStep] | None:
509     steps: list[PlanStep] = []
510     cursor = release
511     last_room: str | None = None
512     for position, task in enumerate(ordered_tasks, start=1):
513         task_release = max(cursor, task.bladder_ready_dt) if task.bladder else cursor
514         found = calendar.find_task(
515             task,
516             task_release,
517             latest_end,
518             last_room,
519             room_override=room_override,
520         )
521         if found is None:
522             for step in reversed(steps):
523                 calendar.release(step)
524             return None
525         start_index, room_id, transfer_slots = found
526         step = PlanStep(task, start_index, room_id, position, transfer_slots)
527         calendar.reserve(step)
528         steps.append(step)
529         cursor = calendar.end_timestamp(start_index, task.duration_slots)
530         last_room = room_id
531     return steps

```

Listing 7 共同设备优先的患者级排程实际源码

```

534 def plan_event(
535     calendar: WorkCalendar,
536     event_tasks: list[Task],
537     release: pd.Timestamp,
538     latest_end: pd.Timestamp,
539 ) -> list[PlanStep] | None:
540     if any(not task.compatible_rooms for task in event_tasks):
541         return None
542     orders = task_order_candidates(event_tasks)
543     common = set(event_tasks[0].compatible_rooms)
544     for task in event_tasks[1:]:
545         common &= set(task.compatible_rooms)
546     if common:
547         candidate_rooms = sorted(
548             common,
549             key=calendar.room_priority_key,
550         )[:3]
551         for room in candidate_rooms:
552             steps = tentative_plan(calendar, orders[0], release, latest_end, room)
553             if steps is not None:
554                 return steps
555     for ordered in orders:
556         steps = tentative_plan(calendar, ordered, release, latest_end, None)
557         if steps is not None:

```

```

558         return steps
559     return None

```

附录 E 问题三程序

问题三的正式下限按全评价期背景事件总数计算。`build_p3_candidate` 先生成住院优先基线；若其未达到下限，则从空日历执行 `global_deficit_repair` 分支，先排至全期背景目标，再排住院和剩余背景。`solve_p3_epsilon` 仅从达到下限的候选中按住院完成数及等待指标选择。下面直接摘录 `run_pre_freeze_scenarios.py` 中的两个实际函数。

Listing 8 住院基线与全局缺口重构分支的实际源码

```

216
217 def build_p3_candidate(
218     events: pd.DataFrame,
219     tasks: dict[str, list[scheduler.Task]],
220     capacity: pd.DataFrame,
221     rooms: list[str],
222     room_names: dict[str, str],
223     policy: str,
224     capacity_column: str,
225     transfer_minutes: int,
226     required_background: int,
227     construction_method: str,
228     background_strategy: str,
229 ) -> dict[str, Any]:
230     """Construct one joint solution around the shared P2 scheduling kernel."""
231     calendar = scheduler.create_calendar(
232         capacity, rooms, policy, capacity_column, transfer_minutes,
233     )
234     lookup = events.set_index("event_id")
235     if construction_method == "inpatient_first":
236         _, inpatient_plans, inpatient_failure = scheduler.run_group_with_policy(
237             calendar, events, tasks, policy, "inpatient",
238         )
239         _, background_plans, background_failure = scheduler.run_group_with_policy(
240             calendar, events, tasks, policy, "background",
241             strategy_override=background_strategy,
242         )
243         reservation = {
244             "formal_constraint_scope": "entire evaluation year",
245             "daily_quota_used": False,
246             "required_background_events": required_background,
247             "reserved_background_events_before_inpatient": 0,
248             "background_attempts_before_inpatient": 0,
249             "reservation_target_met": required_background == 0,
250         }
251     elif construction_method == "global_deficit_repair":
252         background_order = scheduler.event_priority(
253             events, tasks, "background", background_strategy,
254         )
255         reserved_plans, reserved_failure, remaining, reservation = (
256             schedule_background_until_target(
257                 calendar, background_order, lookup, tasks, required_background,
258             )
259         )
260         _, inpatient_plans, inpatient_failure = scheduler.run_group_with_policy(
261             calendar, events, tasks, policy, "inpatient",
262         )
263         residual_plans, residual_failure = scheduler.schedule_group(
264             calendar, remaining, lookup, tasks,

```

```

265     )
266     background_plans = {**reserved_plans, **residual_plans}
267     background_failure = {**reserved_failure, **residual_failure}
268     else:
269         raise ValueError(f"unknown P3 construction method: {construction_method}")
270
271     plans = {**background_plans, **inpatient_plans}
272     failure = {**background_failure, **inpatient_failure}
273     schedule = scheduler.plan_to_schedule(calendar, plans, events, policy, room_names)
274     outcomes = scheduler.outcomes_from_schedule(events, schedule, failure, policy)
275     metrics = scheduler.metrics_from_outcomes(
276         outcomes, schedule, calendar, policy, "P3_CANDIDATE",
277     )
278     background_completed = int(metrics["background_on_planned_day"])
279     diagnostics = {
280         **reservation,
281         "construction_method": construction_method,
282         "background_order_strategy": background_strategy,
283         "background_completed_events": background_completed,
284         "background_deficit_events": max(0, required_background - background_completed
285     ),
286         "epsilon_constraint_satisfied": background_completed >= required_background,
287         "inpatient_completed_events": int(metrics["inpatient_complete_48h"]),
288     }
289     return {
290         "schedule": schedule,
291         "outcomes": outcomes,
292         "calendar": calendar,
293         "metrics": metrics,
294         "diagnostics": diagnostics,

```

Listing 9 全期背景下限与候选选择的实际源码

```

305
306 def solve_p3_epsilon(
307     events: pd.DataFrame,
308     tasks: dict[str, list[scheduler.Task]],
309     capacity: pd.DataFrame,
310     rooms: list[str],
311     room_names: dict[str, str],
312     policy: str,
313     alpha: float,
314     capacity_column: str = "capacity_main",
315     transfer_minutes: int = scheduler.TRANSFER_MINUTES,
316 ) -> dict[str, Any]:
317     """Run the production P3 heuristic on a full year or benchmark subset."""
318     if not 0.0 <= alpha <= 1.0:
319         raise ValueError("alpha must lie in [0, 1]")
320     background_mask = events["evaluation_cohort"] & events["mandatory_background"]
321     total_background = int(background_mask.sum())
322     required_background = int(math.ceil(alpha * total_background))
323     baseline = build_p3_candidate(
324         events, tasks, capacity, rooms, room_names, policy, capacity_column,
325         transfer_minutes, required_background, "inpatient_first", "scarcity",
326     )
327     candidates = [baseline]
328     repair_triggered = not baseline["diagnostics"]["epsilon_constraint_satisfied"]
329     if repair_triggered:
330         def build_repair(background_strategy: str) -> dict[str, Any]:
331             return build_p3_candidate(
332                 events, tasks, capacity, rooms, room_names, policy, capacity_column,
333                 transfer_minutes, required_background, "global_deficit_repair",
334                 background_strategy,
335             )

```

```

336     # The two repair calendars are independent and read only the shared
337     # event/task inputs, so they can use separate CPU cores safely.
338     with ThreadPoolExecutor(max_workers=2) as pool:
339         candidates.extend(pool.map(build_repair, ("scarcity", "fcfs")))
340     feasible_candidates = [
341         candidate for candidate in candidates
342         if candidate["diagnostics"]["epsilon_constraint_satisfied"]
343     ]
344     if feasible_candidates:
345         selected = min(feasible_candidates, key=p3_candidate_key)
346     else:
347         selected = min(
348             candidates,
349             key=lambda candidate: (
350                 -candidate["diagnostics"]["background_completed_events"],
351                 *p3_candidate_key(candidate),
352             ),
353         )
354     return {
355         "selected": selected,
356         "candidates": candidates,
357     }

```

附录 F 模型验证程序

验证程序从导出的任务表重新建立区间和容量占用，不调用主调度器的插入函数。除逐任务检查释放、截止、准备和设备能力外，还独立检查设备与患者区间重叠、跨室转运以及每个 5 分钟槽的医生并发。下面直接摘录 `verify_unified_schedule.py` 中的互斥与容量复核；问题二、三精确子问题分别见两个 CP-SAT 程序。

Listing 10 互斥、转运与医生容量复核的实际源码

```

220 event_meta = events.set_index("event_id")
221 scheduled_patient = merged["event_id"].map(event_meta["patient_id"]).astype(str)
222 record(
223     "scheduled_patient_matches_input",
224     int(merged["patient_id"].astype(str).ne(scheduled_patient).sum()),
225 )
226 merged["special_case_input"] = merged["event_id"].map(event_meta["special_case"])
227 merged["expected_duration_minutes"] = np.where(
228     merged["special_case_input"],
229     merged["duration_upper_min"],
230     merged["duration_nominal_min"],
231 )
232 merged["expected_duration_minutes"] = (
233     np.ceil(merged["expected_duration_minutes"].astype(float) / SLOT_MINUTES)
234     .clip(lower=1)
235     * SLOT_MINUTES
236 )
237 duration_bad = (
238     merged["duration_minutes"].ne(merged["expected_duration_minutes"])
239     | (merged["end_dt"] - merged["start_dt"]).dt.total_seconds().div(60).ne(
240         merged["duration_minutes"]
241     )
242 )
243 record("duration_matches_5pct_scenario", int(duration_bad.sum()))
244
245 compat_bad = [
246     room not in room_set(rooms)
247     for room, rooms in zip(
248         merged["room_id"].astype(str),
249         np.where(merged["bedside_input"], merged["bedside_compatible_rooms"], merged["
250 project_compatible_rooms"]),
251     )
252 ]

```

```

253 record("project_room_compatibility", int(sum(compat_bad)))
254 record(
255     "bedside_uses_room_7",
256     int((merged["bedside_input"] & merged["room_id"].ne("7")).sum()),
257 )
258
259 release_column = "base_release_dt" if args.preparation_mode == "item" else "release_dt"
260 release = schedule["event_id"].map(event_meta[release_column])
261 deadline = schedule["event_id"].map(event_meta["deadline_dt"])
262 record("task_not_before_release", int(schedule["start_dt"].lt(release).sum()))
263 record("task_complete_by_event_deadline", int(schedule["end_dt"].gt(deadline).sum()))
264 bladder_ready = schedule["event_id"].map(pd.to_datetime(event_meta["bladder_ready_dt"]))
265 record(
266     "bladder_task_after_readiness",
267     int((merged["bladder_input"] & merged["start_dt"].lt(bladder_ready.to_numpy()))).sum()
268 ),
269 record(
270     "five_minute_grid",
271     int(
272         (
273             schedule["start_dt"].dt.minute.mod(SLOT_MINUTES).ne(0)
274             | schedule["end_dt"].dt.minute.mod(SLOT_MINUTES).ne(0)
275             | schedule["start_dt"].dt.second.ne(0)
276             | schedule["end_dt"].dt.second.ne(0)
277         ).sum()
278     ),
279 )
280
281 block_bad = 0
282 for row in schedule[["start_dt", "end_dt"]].itertuples(index=False):
283     morning = row.start_dt.hour < 12 and row.end_dt <= row.start_dt.normalize() + pd.
284         Timedelta(hours=12)
285     afternoon = row.start_dt.hour >= 13 and row.end_dt <= row.start_dt.normalize() + pd.
286         Timedelta(hours=17)
287     if row.start_dt.normalize() != row.end_dt.normalize() or not (morning or afternoon)
288     :
289         block_bad += 1
290 record("inside_single_work_block", block_bad)
291
292 fasting_bad = merged["fasting_input"] & merged["end_dt"].gt(
293     merged["start_dt"].dt.normalize() + pd.Timedelta(hours=10)
294 )
295 record("fasting_complete_by_10", int(fasting_bad.sum()))
296
297 room_overlap = 0
298 for _, group in schedule.sort_values(["room_id", "start_dt"]).groupby("room_id", sort=
299     False):
300     room_overlap += int(group["start_dt"].iloc[1:].reset_index(drop=True).lt(
301         group["end_dt"].iloc[:-1].reset_index(drop=True)
302     ).sum())
303 record("no_room_overlap", room_overlap)
304
305 event_overlap = 0
306 transfer_bad = 0
307 sequence_bad = 0
308 for _, group in schedule.sort_values(["event_id", "sequence_position"]).groupby("
309     event_id", sort=False):
310     expected_sequence = np.arange(1, len(group) + 1)
311     sequence_bad += int((group["sequence_position"].to_numpy() != expected_sequence).
312         sum())
313     if len(group) < 2:
314         continue
315     starts = group["start_dt"].iloc[1:].reset_index(drop=True)
316     ends = group["end_dt"].iloc[:-1].reset_index(drop=True)
317     prior_rooms = group["room_id"].iloc[:-1].reset_index(drop=True)
318     current_rooms = group["room_id"].iloc[1:].reset_index(drop=True)
319     switches = current_rooms.ne(prior_rooms)
320     event_overlap += int(starts.lt(ends).sum())
321     transfer_bad += int((switches & starts.lt(ends + pd.Timedelta(minutes=args.
322         transfer_minutes))).sum())
323 record("contiguous_event_sequence_numbers", sequence_bad)

```

```

317 record("no_same_event_task_overlap", event_overlap)
318 record(
319     "within_event_cross_room_transfer_at_least_configured_minutes",
320     transfer_bad,
321     {"transfer_minutes": args.transfer_minutes},
322 )
323
324 patient_overlap, patient_transfer_bad = patient_timeline_violations(
325     schedule, args.transfer_minutes,
326 )
327 record("no_patient_task_overlap_across_events", patient_overlap)
328 record(
329     "adjacent_patient_cross_room_transfer_at_least_configured_minutes",
330     patient_transfer_bad,
331     {"transfer_minutes": args.transfer_minutes},
332 )
333
334 # Reconstruct doctor concurrency by expanding each scheduled interval to
335 # its occupied 5-minute starts, then compare with weekday-slot capacity.
336 occupied_parts: list[pd.DataFrame] = []
337 for slots, group in schedule.groupby(schedule["duration_minutes"].floordiv(SLOT_MINUTES)
338                                     ).astype(int)):
339     base = group[["start_dt"]].reset_index(drop=True)
340     repeated = pd.DataFrame(
341         {
342             "slot_dt": np.concatenate(
343                 [
344                     (base["start_dt"] + pd.Timedelta(minutes=offset * SLOT_MINUTES)).
345                     to_numpy()
346                     for offset in range(int(slots))
347                 ]
348             )
349         }
350     )
351     occupied_parts.append(repeated)
352     occupied = pd.concat(occupied_parts, ignore_index=True)
353     load = occupied.groupby("slot_dt", as_index=False).size().rename(columns={"size": "
354     doctor_load"})
355     load["weekday"] = load["slot_dt"].dt.dayofweek
356     load["slot_index"] = load["slot_dt"].map(slot_index)
357     cap = capacity[["weekday", "slot_index", "capacity_main"]]
358     load = load.merge(cap, on=["weekday", "slot_index"], how="left", validate="many_to_one"
359 )
360 record("doctor_capacity_defined", int(load["capacity_main"].isna().sum()))
361 record(
362     "doctor_concurrency_within_capacity",
363     int(load["doctor_load"].gt(load["capacity_main"]).sum()),
364     {
365         "max_observed_load": int(load["doctor_load"].max()),
366         "max_capacity": int(load["capacity_main"].max()),
367     },
368 )

```